# zkSketch: Verifiable and Privacy-Preserving Sketch-Based Streaming Analytics via Zero-Knowledge Proofs

Binbin Gu
Nanyang Technological University
Singapore
binbin.gu@ntu.edu.sg

Wei Dong
Nanyang Technological University
Singapore
wei_dong@ntu.edu.sg

## ABSTRACT

Using sketches to summarize data streams is a standard technique to balance computational and space efficiency against estimation accuracy. Ideally, every released result should also be verifiable while revealing nothing beyond the result itself. Zero-knowledge proofs (ZKPs) achieve this: the curator publishes a commitment that serves as the identity of the hidden stream and a proof that each released result is correctly computed from the committed data. However, existing ZKP systems for data analytics either focus on static datasets, prove each result over only a bounded window of a stream, or prove a fresh summary per epoch and merge the queried epochs inside every query proof, so that query proving cost grows linearly with the number of epochs queried. This motivates us to develop a sketch-based ZKP framework for verifiable and privacy-preserving stream processing. The central challenge is to ensure that the proof remains compact and can be updated efficiently, thereby preserving the computational and space-efficiency advantages of sketches.

In this paper, we propose zkSketch, a sketch-based ZKP framework that supports the streaming model. To keep the proof compact, zkSketch recursively composes the per-batch proofs into a single proof. To prove each sketch update efficiently, zkSketch certifies the batch by its *effect* on the sketch instead of scanning the sketch state for every record inside the circuit. We instantiate zkSketch for six sketches: the amortized per-update circuit size is $\Theta(d)$ for Count-Min and $\Theta(k)$ for the Bloom filter, with $d$ and $k$ hash functions, $\Theta(1)$ for HyperLogLog, k-minimum values, and Misra–Gries, and $\Theta(\log s')$ for Greenwald–Khanna with summary capacity $s'$. In experiments, the per-update proving time stays flat as the stream grows, and the single proof of the whole maintenance history verifies in around 15 ms regardless of the stream length.

**PVLDB Reference Format:**
Binbin Gu and Wei Dong. zkSketch: Verifiable and Privacy-Preserving Sketch-Based Streaming Analytics via Zero-Knowledge Proofs. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

**PVLDB Artifact Availability:**
The source code, data, and/or other artifacts have been made available at https://github.com/tuzijun111/zkSketch.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

## 1 INTRODUCTION

Streaming algorithms are widely used in domains such as finance, network monitoring, and public health, where data must be processed continuously under strict resource constraints. A key tool in stream processing is the sketch [2], a compact data structure that offers a practical trade-off between computational efficiency and estimation accuracy. In this model, a curator receives updates, maintains the sketch, and answers queries. A central challenge is ensuring that these answers are reliable without requiring users to place complete trust in the curator: the correctness of its sketch maintenance and query computation should be independently *verifiable*. This is particularly important because sketches inherently produce approximate results, making it difficult to distinguish legitimate approximation error from incorrect or malicious computation. At the same time, such verification must be *privacy-preserving*, since data streams often contain sensitive information and the curator may wish to reveal nothing beyond the authorized query outputs.

Verifying stream statistics requires evidence that can be independently checked without revealing the underlying private logs [74]. Trusted execution environments (TEEs) can protect the computation and attest to the code being run [53, 72], but their guarantees depend on the hardware and its attestation infrastructure. We focus on cryptographic verification through zero-knowledge proofs (ZKPs). At a high level, the curator acts as the *prover*: it proves that its sketch updates and answers follow the declared rules on records fixed by a public *commitment*, while keeping the records and sketch state private. Any auditor can check this proof without trusting the execution platform. Proof generation adds computation, so we target applications that can tolerate certification delay: proofs are generated for closed batches while new records continue to arrive.

Although ZKPs have been increasingly explored for verifiable analytics, most existing systems focus on computations over static datasets [18, 34, 43, 58, 66, 70]. Some ZKP-based works that consider streaming data prove each released result over only a bounded portion of the stream, such as a window of recent records or a selected sub-stream [27, 62]. This setting differs from the conventional streaming model, in which query results are continuously released over all data observed in the stream up to the current time. zkAnalytics [74] uses a zero-knowledge virtual machine (zkVM) to certify a separate summary for a group of incoming batches. However, proving the computation of each summary incurs the high overhead of checking its instructions and memory accesses.

This motivates us to develop an efficient ZKP framework for commonly used sketching algorithms. To fully preserve the advantages of sketches, the proof should also support incremental updates and compactly summarize the entire computation history, just as a sketch compactly summarizes the stream. Naturally, the

proof consists of two components: *correct update*, which certifies that the sketch is maintained correctly as new records arrive, and *correct query answering*, which certifies that each released answer is correctly computed from the maintained sketch. Since updates occur continuously and typically far more frequently than queries, proving sketch updates dominates the cost. We therefore focus on keeping the accumulated proof compact while ensuring that each update can be proved efficiently.

**Challenge of keeping the proof compact.** To support the verification of continuous query answering over a data stream, a straightforward approach is to have the prover construct, at each time step, a new proof from scratch over the entire stream observed so far. However, this approach is prohibitively expensive. A more efficient approach is to prove sketch updates incrementally, which we call the *per-batch proof chain*: rather than re-proving the entire computation history, when a new batch of records arrives, the prover proves only the transition from the previously committed sketch state to the updated state. These per-batch proofs are simply chained one after another through the shared sketch states, without any further combination. The prover's work at each time step is therefore fixed by the batch capacity and the sketch size, independent of the stream length. However, the growing cost now shifts to the verifier: since each proof certifies only its own transition, it must check all $t$ transition proofs accumulated after $t$ batches, together with the $t-1$ boundary links between them, a cost that again grows with the stream. Therefore, we need to compress the per-batch proof chain into a single proof whose proof-checking cost is independent of the stream length.

**Challenge of proving sketch updates efficiently.** For each batch of data updates, the prover must show that the old sketch state, after processing the updates, becomes the new sketch state. Under the ZKP framework, proof generation is done by formulating the relationship between the input and the output as a circuit. Because the circuit is fixed and published before the stream arrives, its size and structure must be independent of the private data. Otherwise, its structure could reveal which sketch cells are updated or which summary entries are retained. A brute-force circuit implementation makes every update oblivious by scanning the entire sketch to select or move affected entries. However, for a batch of $m$ records, this requires at least $m$ full scans, repeating the state work for every record. We must therefore redesign the proof of sketch updates to check both cell updates and summary reductions without incurring the cost of replaying each record's state accesses and rearrangements.

## 1.1 Our result

In this paper, we present zkSketch, a sketch-based ZKP framework for streaming processing that maintains one sketch over the whole stream and certifies its entire update history with one composed proof. To deal with the challenge of keeping the proof compact, zkSketch compresses the per-batch proof chain into a *recursive chain* through *recursive proof composition* [11, 40, 63]. More precisely, at each step, the prover establishes two facts: that the proof certifying the computation history up to the previous step is valid, and that the batch proof of the current update is valid. Consequently, each new proof recursively certifies the entire computation history, allowing the verifier to check only the most recent proof at a proof-checking cost independent of the stream length. However, this recursive chain introduces a sequential dependency among compositions: each composition can start only after the preceding one is complete, so composing $t$ batch proofs takes $t-1$ steps in sequence. Since updates occur far more frequently than queries, multiple batches typically arrive between two consecutive queries, yet a strictly sequential composition chain fails to exploit the parallelism available across these batches. The batch proofs themselves can still be generated independently, as in the per-batch proof chain. Therefore, we organize the proof composition as a *proof forest* (Figure 2) to compose the batch proofs in parallel as well, while keeping the compactness of the recursive chain. The proof-generation procedure has two stages. First, zkSketch performs ordinary sketch updates in a single plaintext pass to obtain all batch-boundary states, at low cost relative to cryptographic proving. Second, zkSketch independently generates batch-update proofs in parallel and recursively composes them pairwise in a tree into a single compact proof.

To prove each sketch update efficiently, our key idea is that the circuit does not need to replay the update program step by step; it suffices to verify the batch's overall *effect* on the sketch. Therefore, zkSketch lets the prover compute the batch update outside the circuit and supply intermediate tables as a bounded *batch-effect certificate*. The circuit then checks that these tables are consistent with the committed batch and carry the old committed state into the new one, sharing the state work across records and avoiding repeated scans of the entire sketch. Since the tables are padded to public capacities and the checks follow a fixed public schedule, the circuit's shape reveals nothing about the private records. Specifically, the design follows two patterns: installing *per-cell changes* in indexed arrays with one offline memory-checking balance per batch, as in Count-Min [21] and HyperLogLog [28], and checking *summary reductions* and the placement of surviving entries on sorted copies of the candidates, as in Misra–Gries [1, 47] and Greenwald–Khanna [32].

In summary, this paper makes the following contributions:

- We propose zkSketch, a sketch-based ZKP framework for stream processing. Its proof forest combines batch proofs into one compact history proof whose proof-checking cost is independent of stream length. Batch proofs can be generated and composed in parallel once the boundary states are available, even when sketch updates must follow stream order.
- To efficiently prove sketch updates, we design two circuit patterns that certify the effect of each batch: applying changes to arrays and reducing bounded-size summaries. Together, they support six sketch families. The amortized circuit size per update, for batches as large as the sketch, is $\Theta(d)$ for Count-Min, $\Theta(k)$ for Bloom filters, $\Theta(1)$ for HyperLogLog, k-minimum values (KMV), and Misra–Gries, and $\Theta(\log s')$ for Greenwald–Khanna, where $d$ and $k$ denote the numbers of hash functions and $s'$ is the summary capacity.
- We implement zkSketch in Plonky3 and evaluate all six sketches. Compared with zk-Analytics, the proving time of zkSketch is 23–82× faster. The proof verifies in about 15 ms regardless of the stream length.

## 2 RELATED WORK

Privacy and integrity in stream processing have been studied through several approaches with different trust and cost trade-offs. Differential privacy bounds what continual releases reveal about individual records [17, 19, 25, 39, 41, 69, 71], but does not by itself certify that the curator executed the declared mechanism. Trusted execution environments (TEEs) protect stream and database processing inside hardware enclaves [4, 36, 53, 57, 72, 73], while placing trust in the hardware, attestation keys, and vendor infrastructure [37, 64, 65]. Secure multiparty computation and secure aggregation distribute trust among non-colluding parties [6, 14, 15, 22, 38, 48, 59, 68]; encrypted stream stores instead restrict access through keys [16]. Their privacy guarantees do not automatically provide a transferable proof for an outside verifier. Publicly auditable MPC provides external verification, but its auditing cost grows with the computation [7]. Authenticated stream processing offers integrity through an owner that signs or monitors the stream, without generally hiding the records used in verification [29, 42, 51, 52, 67].

More closely related to our work are ZKP-based approaches that certify computations over private data streams. SPHinx [27] proves statistics over selected portions of a signed stream, mainly sliding windows. zkStream [62] proves individual operators of a windowed stream program and passes committed values between proofs. Neither system instantiates and evaluates a committed summary of the accumulated stream, so extending its evaluated design to the whole prefix would require recomputation or a growing sequence of proof checks. zk-Analytics [74] uses a zero-knowledge virtual machine to certify a separate summary for each *epoch*, a group of incoming log batches. For each query, its prover merges the summaries of the selected epochs and proves the query on the merged summary. The verifier must check every selected epoch's aggregation proof as well as the query proof. As queries cover a longer stream prefix, the prover processes more summaries and the verifier checks more aggregation proofs. In contrast, zkSketch maintains a compact sketch across batches and answers queries directly from the accumulated state. Its specialized circuits check each batch's effect on the sketch, while the proof forest combines the batch proofs into one history proof. The verifier then checks one history proof and one query proof instead of a separate proof for every batch. The verifier still checks these proofs against a public log of batch commitments (Section 4.2).

## 3 PRELIMINARIES

We review the zero-knowledge proof tools and streaming sketches used in zkSketch.

### 3.1 Zero-Knowledge Proofs

A *zero-knowledge proof* (ZKP) [30] enables a *prover* to convince a *verifier* that a public statement is true while keeping the supporting data private. The statement includes the public inputs and any revealed output of a computation. In this work, we use *zero-knowledge STARKs* [9] built with Plonky3 [55]. These proofs require no trusted setup and provide three standard guarantees:

- *Completeness.* An honest prover can prove a true statement.
- *Soundness.* A dishonest prover cannot prove a false statement except with negligible probability.
- *Zero knowledge.* The verifier learns nothing beyond the statement.

At a high level, a zero-knowledge proof establishes that there exists a private *witness* satisfying a set of public constraints for a given public statement. The witness contains the hidden information needed to demonstrate the statement's validity, which may include secret inputs, intermediate values, or an execution trace of the computation being verified. To generate a proof, the target computation is encoded as an arithmetic circuit that expresses the constraints to be satisfied by the witness. The circuit, together with the public statement and private witness, is then provided to a ZKP system to generate a proof. The verifier checks the proof against the statement and a short public *verification key*, a digest of the circuit, so the key fixes which computation the proof certifies.

**Circuits.** An arithmetic circuit encodes a computation as algebraic constraints involving simple operations, such as addition and multiplication [34]. For example, consider updating a counter from $c$ to $c'$ by adding an incoming value $v$. The circuit enforces the constraint $c + v - c' = 0$. The prover supplies these values as part of the witness, and a valid witness must satisfy this equation. The constraint remains the same regardless of the values of $c$ and $v$. More generally, a circuit is *oblivious* if its size and structure depend only on public parameters, preventing its structure from revealing information about private inputs.

**Remark 1.** The prover computes the results and any required intermediate values outside the circuit and supplies them as part of the witness. Our circuits then check that these values satisfy the required relationships with the inputs, rather than reproduce the steps used to compute them.

**Commitments.** To ensure that a proof concerns a particular input, we use cryptographic commitments. To commit to $x$, a party uses private randomness to produce a public commitment $\mathsf{Com}(x)$, also called a *fingerprint*. It can later *open* this commitment by providing $x$ and the randomness for verification. A secure scheme is *hiding*, so the commitment conceals $x$, and *binding*, so the party cannot feasibly open it to a different value. Specifically, our commitments hash the data together with secret randomness. The proof checks the opening internally, allowing the verifier to confirm that the computation uses the committed data while keeping the data and randomness private.

**ZKP backend.** Our system uses Plonky3 [55], a toolkit for constructing proof systems, to generate zero-knowledge STARKs [9]. We express circuit checks as polynomial constraints over witness tables through an *algebraic intermediate representation* (AIR). Plonky3 also supports post-quantum security.

**Proving cost.** Proof generation is the main computational bottleneck: the prover must process the witness tables and their constraints. We measure circuit size by the total number $G$ of *circuit rows* across these tables, whose entries lie in a prime field $\mathbb{F}$. With field and table widths and proof parameters fixed, the backend performs $O(G \log G)$ polynomial work. Thus, proving time grows with circuit size, even when the underlying sketch operation is inexpensive. As a result, reducing $G$ is central to making proof generation efficient.

## 3.2 Streaming Sketches

A *sketch* is a compact data structure that summarizes a dataset to support approximate queries without storing all records. Its parameters determine the trade-off between memory usage and query accuracy. We focus on *streaming sketches*, which incrementally update their summaries as records arrive without retaining the full stream. Unlike task-specific summaries, sketches can support queries that are not known in advance. For example, a single exact counter suffices to track the frequency of a predefined key $A$, but cannot answer frequency queries for other keys. In contrast, a Count-Min sketch [21] maintains a compact summary that supports frequency estimates for arbitrary keys, even when the queried keys are chosen after the records arrive.

# 4 SYSTEM OVERVIEW

We first define the streaming model, then describe how zkSketch certifies sketch updates and query answers.

## 4.1 Streaming Model

We model execution as a potentially unbounded sequence of events $e_1, e_2, \ldots$, each either a private *record event* or a public *query event*. Let $x_1, x_2, \ldots$ denote records in arrival order and $N(j)$ the number preceding event $e_j$; $N$ denotes the total seen so far. A query $q_j$ at event $j$ applies the sketch query function $\mathsf{Query}_{q_j}$ to the state obtained from $x_1, \ldots, x_{N(j)}$ by the declared batch rule below, starting at the public empty state $S_0$. Thus its answer concerns the whole preceding prefix, with the approximation guarantee of the sketch.

For efficient maintenance, consecutive records form batches $B_t = (x_{N_{t-1}+1}, \ldots, x_{N_t})$ of at most $m$ records. Here $t$ is the *version*, $N_t$ counts all real records through that version, and $S_t = \mathsf{UpdateBatch}(S_{t-1}, B_t)$ is the declared deterministic batch rule. The deployment fixes a public *profile*: its sketch family, update and query rules, commitment hash, capacities, composition schedule, and bounds on $N$ and on the total weight of the records. Its key hashes are fixed public functions with no parameter chosen by the operator. So that each answer covers all preceding records, an honest prover seals (closes) a batch when it reaches $m$ records or a query arrives. A partial batch is padded to $m$ with dummies, after which $B_t$ denotes the padded batch. A query with no intervening records reuses the current version. Consequently, $q_j$ is answered at $\mathrm{ver}(j)$, the largest $t$ with $N_t = N(j)$, taking $N_0 = 0$.

## 4.2 Workflow and Problem Statement

With the stream prefix defined, we now describe how an answer is certified. Figure 1 connects four parties. The *data source* supplies authentic records to the untrusted operator (the curator of Section 1), which acts as the *prover* and maintains the sketch. A *log service*, such as a transparency log [23], maintains an upload log ULog and a query log in a consistent append-only order. Any *verifier*, including the auditor, can use these public logs and the published proofs to check an answer.

**Committing batches.** Recall that a batch closes when it fills or a query arrives. The prover pads $B_t$, computes $S_t$, and appends $\mathsf{Com}(B_t)$ and $\mathsf{Com}(S_t)$ to ULog. The log begins with $\mathsf{Com}(S_0)$, computed from the public empty state. The log service records a

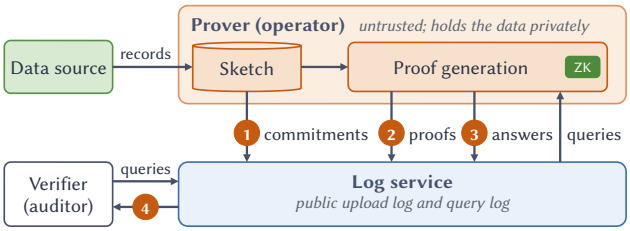



**Figure 1: The parties and public information used to verify a sketch answer. The prover logs commitments (1) and publishes proofs (2) and answers (3) through the log service, from which the verifier reads them (4).**

query after the commitments of any batch it closes and before any later batch, so its position fixes its version $t = \mathrm{ver}(j)$.

**Proving updates.** Once the commitments are fixed, the prover publishes a *batch proof* $\pi_t$ certifying the update from $S_{t-1}$ to $S_t$ using $B_t$. To keep the accumulated proof compact, the prover combines the batch proofs into a single *history proof* $\pi_h$, which certifies all updates from $S_0$ to $S_t$.

**Answering queries.** For query $q_j$, the prover evaluates the declared query function on $S_t$ and returns the answer $res_j$ with a *query proof* $\pi_q$. This proof binds the query and its answer to $\mathsf{Com}(S_t)$, using the query circuits.

**Public interface and proved statement.** Given a query $q_j$ at logged version $t = \mathrm{ver}(j)$, the prover publishes $res_j$ and $\mathsf{Com}(S_t)$ through the log service, with the history proof $\pi_h$ and the query proof $\pi_q$. The verifier accepts if

$$\mathsf{Verify}\big(\mathsf{Com}(S_0), \mathsf{Com}(S_t), \mathsf{ULog}, \pi_h\big) = 1,$$
$$\mathsf{Verify}\big(\mathsf{Com}(S_t), q_j, res_j, \pi_q\big) = 1.$$

The first check also matches the batch range and commitments of the history proof to ULog through version $t$, and both checks use the same $\mathsf{Com}(S_t)$. If both pass, the verifier is assured that $res_j = \mathsf{Query}_{q_j}(S_t)$, where $S_t$ results from applying the logged batches $B_1, \ldots, B_t$ to $S_0$ in order.

Recall that proving a circuit with $G$ rows requires $O(G \log G)$ polynomial work (Section 3). Therefore, we focus on reducing circuit size to prove sketch updates and queries efficiently.

# 5 THREAT MODEL AND SECURITY GUARANTEES

**Threat model.** We consider a mutually distrustful setting between the prover and the verifier, which gives rise to three threats: (T1) *wrong input*, through altered, omitted, reordered, or replayed committed batches; (T2) *wrong result*, through a different update rule, a state older than the logged version, or an incorrect query answer; and (T3) a *curious verifier*, seeking information beyond the intended releases.

**Security guarantees.** For (T1), binding commitments identify the batch contents, and verification against the ordered log identifies the prefix. For (T2), we rely on three assumptions: the soundness of the Plonky3 backend, which takes our circuits as input and which we assume carries through recursive verification (Appendix A.1);

the binding of our hash-based commitments; and, for Count-Min and Misra–Gries, the profile's bound on the total weight of the records (Appendices B.1.1 and B.4). Each batch proof sees only its batch and the commitments to its two boundary states, so binding is what makes consecutive proofs refer to the same intermediate state. The remaining task is therefore to ensure that the circuits enforce precisely the declared computation, a property we establish in the following sections. For (T3), since the circuit is publicly visible, we require it to be oblivious: its size and structure must not reveal private data. Under this condition, the zero-knowledge property of our zkSTARK backend ensures that the proof reveals nothing beyond the public inputs and query results.

**Scope.** Two orthogonal concerns lie outside the scope of zkSketch. (i) *Input authenticity*: a commitment binds the prover to its committed values so an operator could omit or delay records before committing them; binding the logged batches to the source's records requires the data source to authenticate them, for example by signing them [27, 52]. (ii) *Output leakage*: repeated or adaptive releases may leak through the outputs themselves; such leakage can be bounded by composing zkSketch with differential privacy [24].

# 6 STREAMING PROOF COMPOSITION FRAMEWORK

Recall that each batch proof certifies an update between two committed sketch states. As batches accumulate, we need to summarize these proofs so that checking the history does not require a separate proof check for every batch. Our starting point is the standard recursive chain, which combines batch proofs into one history proof but makes composition sequential. We therefore adopt a *proof forest*: once the boundary states are available, batch proofs can be generated and composed in parallel.

## 6.1 From a Per-Batch Proof Chain to a Single Proof

Each closed batch should be certifiable on its own; the question is how much work this leaves to the verifier. The most direct strategy is one ZKP per batch. When $B_t$ closes, the prover generates a batch proof $\pi_t$ of $S_t = \mathsf{UpdateBatch}(S_{t-1}, B_t)$. The proof reveals only the version and the commitments to the batch and to the old and new states. Consecutive proofs share a state commitment, forming a *per-batch proof chain* [62] (Figure 2a).

Once a batch and its boundary states are available, its proof can be generated independently, at a cost fixed by the batch capacity and sketch parameters. However, auditing version $t$ requires checking all $t$ proofs and their $t-1$ shared-state links. This $\Theta(t)$ verification work motivates a composition layer that summarizes the growing chain in one proof. To do so, we use *recursive proof composition* [11, 63]. At a high level, proof verification is itself a computation that a circuit can check, allowing a new proof to certify earlier proofs. For a fixed proof system, this check costs the same regardless of how much history they cover. We represent the resulting object as an *interval proof*, which certifies a consecutive range of batches.

**Definition 6.1** (Interval proof)**.** An interval proof $\Pi_{[i,j]}$ certifies that applying $B_i, \ldots, B_j$ in order under the declared update rule carries the state committed by $\mathsf{Com}(S_{i-1})$ to that committed by $\mathsf{Com}(S_j)$. Its public label $\mathsf{pub}_{[i,j]}$ consists of these two commitments, the range $(i, j)$, and a *batch hash*, a single hash of the batch commitments $\mathsf{Com}(B_i), \ldots, \mathsf{Com}(B_j)$ that identifies the batches it covers. A batch proof is the case $\pi_t = \Pi_{[t,t]}$, with label $\mathsf{pub}_t = \mathsf{pub}_{[t,t]}$, and $\Pi_{[1,t]}$ is a history proof through version $t$.

With this representation, a *recursive chain* keeps one history proof and extends it one batch at a time (Figure 2b). A *composition step* checks the previous history proof $\Pi_{[1,t-1]}$ and the new batch proof $\pi_t$, checks that they meet at $\mathsf{Com}(S_{t-1})$, and outputs $\Pi_{[1,t]}$. The verifier then needs to check only this one proof against the log (Section 6.2). The prover, however, must finish each composition before starting the next, a sequential dependency of depth $\Theta(t)$. We next show how a proof forest removes this dependency.

## 6.2 Proof Forest Composition

Recall that each batch updates the state left by its predecessor, $S_i = \mathsf{UpdateBatch}(S_{i-1}, B_i)$, so these states must be computed in batch order. Producing a proof, however, involves two stages: *witness generation* computes the states and intermediate values using ordinary plaintext operations, and *proof generation* runs the cryptographic prover on those values. The latter accounts for most of the cost. We therefore compute the boundary states in an inexpensive sequential pass. Once a batch's two boundary states and their commitments are fixed, its remaining witness preparation and proof generation depend only on that batch and those states, not on the preceding batch's proof. Thus, even when sketch updates are sequential, their batch proofs can be generated in parallel.

With the batch proofs independent, the recursive chain's remaining bottleneck is their composition. The key observation is that a composition step needs only two adjacent intervals, neither of which must start at batch one. For example, with four batches, the prover can produce $\Pi_{[1,2]}$ and $\Pi_{[3,4]}$ concurrently, then combine them into $\Pi_{[1,4]}$. This certifies the same history at composition depth two rather than three (Figure 2c). A composition step checks only its two input proofs and where their intervals meet; it does not re-read their batches or re-execute their updates, so its cost does not depend on how much history they cover.

**Definition 6.2** (Composition step)**.** A composition step combines two adjacent interval proofs, its children $\Pi_{[i,h]}$ and $\Pi_{[h+1,j]}$, into $\Pi_{[i,j]}$ through three checks:

(1) **Child proofs.** Both child proofs pass verification.
(2) **Adjacency.** The left child ends at the state commitment where the right child starts, $\mathsf{Com}(S_h)$, and the right child's range starts at batch $h+1$.
(3) **Parent label.** The step outputs $\mathsf{pub}_{[i,j]}$, taking $\mathsf{Com}(S_{i-1})$ and $\mathsf{Com}(S_j)$ from its children and hashing their two batch hashes into one.

To check a history proof $\Pi_{[1,t]}$, a verifier runs one proof check and compares the proof's public label with the log: the range must be $(1, t)$, the outer commitments must be the public $\mathsf{Com}(S_0)$ and the logged $\mathsf{Com}(S_t)$, and the batch hash must equal the one the verifier recomputes from the logged $\mathsf{Com}(B_1), \ldots, \mathsf{Com}(B_t)$ along the profile's composition schedule. This hashing is the only verification work that grows with the history. The sketch logic stays in the batch proofs, so one composition step design serves every

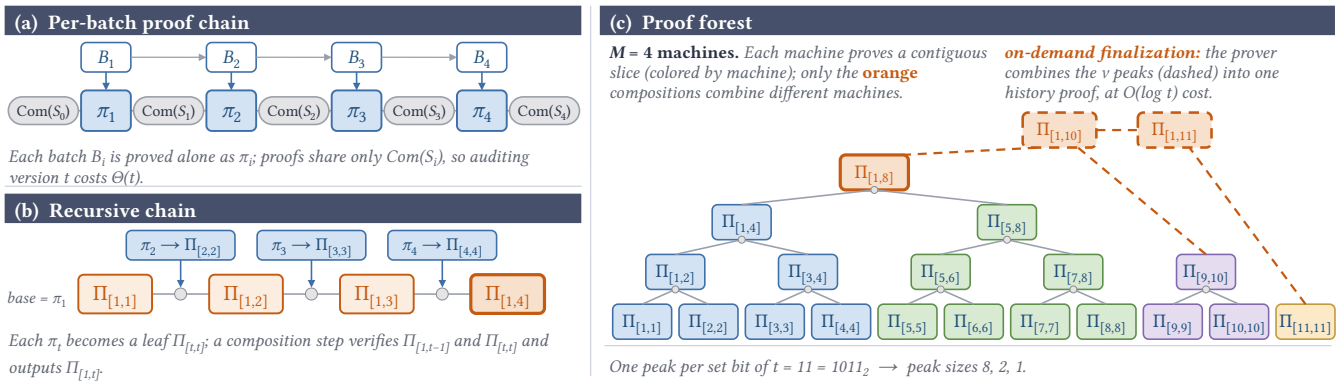



**Figure 2: Proof composition: (a) a per-batch proof chain; (b) a recursive chain; (c) a proof forest at $t = 11$.**

sketch family. It also serves the recursive chain, the forest, and the $M$-machine variant, which differ only in which adjacent intervals they combine. With $M$ machines, each machine builds a forest over a contiguous slice of batches and finalizes it into one root, and $M-1$ further compositions combine their roots.

So far, we can build a tree over any completed interval. A live stream, however, has no fixed endpoint. We therefore maintain a *proof forest* whose roots, called *peaks*, cover the committed prefix. Each new batch proof starts as a size-one peak. Whenever the two rightmost peaks have equal size, we combine them and repeat if the resulting peak matches its left neighbor. This follows the binary-counter schedule of Bentley and Saxe [10] and its use in Valiant's incremental verification [63]. As a result, each peak is a perfect tree over consecutive batches, with at most one peak of each size. At $t = 11 = 8 + 2 + 1$, for example, the peaks are $\Pi_{[1,8]}$, $\Pi_{[9,10]}$, and $\Pi_{[11,11]}$ (Figure 2c). There are generally $\nu \le \lceil \log_2(t+1) \rceil$ peaks. Maintaining them takes $O(\log t)$ compositions per insertion in the worst case and $O(1)$ amortized.

These *live peaks* already allow verification before the stream ends. The prover publishes them as they form. A verifier checks each peak's proof and compares its label with the log, as for a history proof; the peaks' ranges must cover $[1, t]$ consecutively, and adjacent peaks must share their boundary state commitments, from the public $\mathsf{Com}(S_0)$ to the logged $\mathsf{Com}(S_t)$. The query proof is checked against this same $\mathsf{Com}(S_t)$, so answering a query needs no further composition.

When one history proof is needed, the prover *finalizes* the forest by combining its peaks from left to right with $\nu - 1$ further compositions, which yields $\Pi_{[1,t]}$. Building the forest uses $t - \nu$ compositions, so the total is $t - 1$, as in the recursive chain. The benefit is the shorter dependency depth: $O(\log t)$, or five compositions along the longest path at $t = 11$, compared with ten in the chain. Each additional finalization adds up to $\nu - 1$ compositions; our prototype finalizes once, at the end of the stream.

# 7 EFFICIENT CIRCUITS FOR SKETCH UPDATES AND QUERIES

So far, the composition framework has reduced the stream's history to proofs of individual batch updates. We now construct these proofs efficiently. The difficulty is that a sketch accesses cells selected by private keys, whereas a circuit has fixed wiring, so a direct circuit scans the possible cells for every access and pays for the whole state once per record. Our idea is to verify a batch's *combined* effect instead. The prover computes intermediate tables outside the circuit, a *batch-effect certificate*, and the circuit checks their contents and their relation to the committed inputs and output, following certifying algorithms [44]. Every table has a public height, with dummy rows hiding how many rows are in use. We first introduce the four checks from which we build every circuit (Section 7.1), then develop two patterns: installing a batch into an array, on Count-Min (Section 7.2), and reducing a bounded summary, on Misra–Gries (Section 7.3), each followed by its extensions to the other sketches. Section 7.4 covers the query circuits.

## 7.1 Circuit Building Blocks

Before developing the two patterns, we introduce the four building blocks used to construct our circuits.

**Multiset equality.** A *same-rows check* establishes that two tables contain the same complete rows, including repetitions; we write $U' \sim U$ for this relation. We implement this relation with LogUp [35], a lookup argument based on logarithmic derivatives. For tables $U = (u_1, \ldots, u_n)$ and $U' = (u'_1, \ldots, u'_n)$, it checks the randomized identity

$$\sum_{i=1}^{n} \frac{1}{\gamma - \mathrm{enc}(u_i)} = \sum_{i=1}^{n} \frac{1}{\gamma - \mathrm{enc}(u'_i)}.$$

Here enc encodes a complete row, including a label distinguishing separate table relations, and $\gamma$ is a random challenge drawn after the tables are committed.

**Range checks and comparisons.** A range check restricts a value to a declared integer interval. For byte-based checks, we reconstruct the value from bytes and use LogUp to check them against the public table $\{0, \ldots, 255\}$, also bounding any partial top byte. Other checks use Boolean bit decomposition. With suitably bounded operands, comparing $x \le y$ reduces to range-checking $y - x$.

**Sort.** With multiset equality and bounded comparisons, we can verify a sorted copy. The prover sorts the table outside the circuit and supplies the result. A same-rows check preserves its contents,

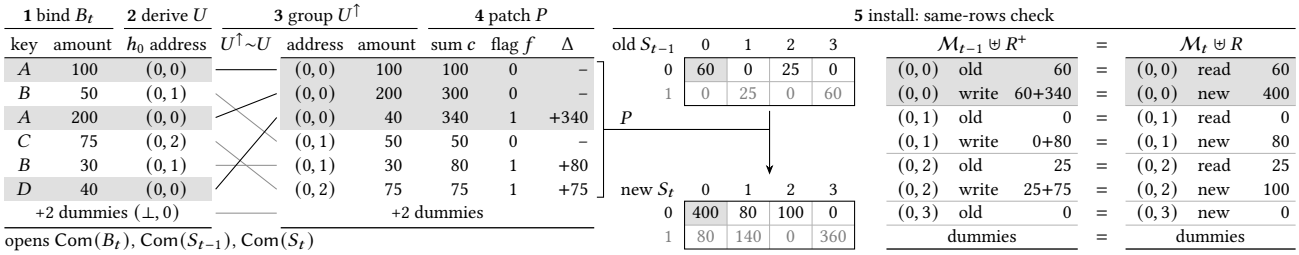



**Figure 3: The five phases of a Count-Min update on sketch row 0 ($d = 2$, $w = 4$, $m = 8$).**

while comparisons between adjacent keys establish its order. Together, these checks form a *sorted-copy check*, which certifies the sorted result without encoding the sorting algorithm.

**Grouped aggregates.** Once equal keys are consecutive, a *group-by check* [34] identifies each group's boundaries and checks a running aggregate within the group. Local arithmetic and comparison constraints fix the sum or maximum, with the last row of each group carrying its result.

All four circuit checks above have a circuit size of $O(n)$ for a table with $n$ rows; Appendix A.2 gives their exact costs, together with those of opening and hashing committed inputs.

## 7.2 Pattern A: Installing a Batch into an Array

*7.2.1 Count-Min and the Key Idea.* Count-Min [21] estimates key frequencies with a $d \times w$ counter array. Each record $(k, v)$, with nonnegative weight $v$, adds $v$ to counter $h_i(k)$ in every row $i$, where the $d$ hashes are public. A query for $k$ returns the smallest of these $d$ counters. Collisions can only increase this estimate, and larger dimensions reduce the error at the cost of a larger state.

**Target.** Let $B_t = ((k_1, v_1), \ldots, (k_m, v_m))$ contain at most $m$ real records, padded with zero-weight dummies, and let $\mathsf{pub}_t$ be its public label (Definition 6.1). The update circuit establish the following.

> **Count-Min update circuit.**
> **Input.** Public label $\mathsf{pub}_t$ and dimensions $m, d, w$; private batch $B_t$, old and new arrays $S_{t-1}, S_t$, their commitment openings, and intermediate tables.
> **Output.** A proof $\pi_t$ certifying the new state commitment.
> **Verification target.** The batch and arrays open to their respective commitments, and every counter satisfies
> $$S_t[i, b] = S_{t-1}[i, b] + \sum_{p:\, h_i(k_p)=b} v_p.$$

**Challenge.** The target uses only additions, but locating their counters is costly. Each address $h_i(k)$ depends on a private key, so a direct circuit selects among the $w$ columns of a sketch row for every access, $\Theta(mdw)$ circuit rows per batch. Grouping repeated keys helps only partly, because different keys may collide at the same counter. The circuit must also guarantee that every counter the batch does not touch stays unchanged.

**Key idea.** We separate the batch's effect from its installation. First, since counters are only added to, the new array depends only on the *net increment* at each address. The circuit can therefore derive these increments from the records and their hashes alone, grouping contributions by address, collisions included, without reading any counter; we call the result a *patch*. Second, installing the patch replaces addressed values, which one *multiset balance* verifies: the old counters together with the *writes*, each touched counter's claimed old value plus its increment, must form the same multiset as the new counters together with the *reads*, the claimed old values. At a changed counter, this ties the old value, the increment, and the new value; elsewhere, it forces the two arrays to agree. This is the offline memory-checking relation [3, 13, 60], and it checks the whole array in one linear pass.

*7.2.2 The Five Phases.* **Running example.** We use a Count-Min array with $d = 2$ rows and $w = 4$ columns, the public hashes $h_0$: $A, D \to 0$, $B \to 1$, $C \to 2$ and $h_1$: $A \to 3$, $B \to 0$, $C, D \to 1$, and the old rows $(60, 0, 25, 0)$ and $(0, 25, 0, 60)$ left by the earlier records $A$:60, $C$:25. The batch contains $A$:100, $B$:50, $A$:200, $C$:75, $B$:30, $D$:40, padded to $m = 8$ with 2 zero-weight dummies $(\perp, 0)$. Figure 3 traces the phases on sketch row 0, where $A$'s two records and $D$'s record together add $100 + 200 + 40 = 340$ to counter $(0, 0)$, while counter $(0, 3)$ receives nothing.

**Phase 1: Binding the batch and states.** The circuit opens $B_t$, $S_{t-1}$, and $S_t$ against $\mathsf{pub}_t$, so that all later checks refer to the committed inputs and output.

**Phase 2: Deriving the contributions.** For each record $p$ and sketch row $i$, the circuit computes the hash itself and forms the contribution

$$U_{p,i} = (a_{p,i},\ v_p), \qquad a_{p,i} = iw + h_i(k_p),$$

so that every counter $S[i, b]$ has a distinct address $a = iw + b$, displayed as $(i, b)$; we write $S[a]$ for the counter at address $a$. A record's destinations therefore cannot be misreported. The table $U$ has $md = 16$ rows, 4 of them from dummies, of amount 0. In row 0, the records of $A$ and $D$ all reach $(0, 0)$.

**Phase 3: Grouping equal addresses.** Contributions to one counter may be scattered across $U$. The prover supplies a copy $U^{\uparrow}$ of $U$ sorted by address, with rows $(a_z, v_z^{\uparrow})$ for $z = 1, \ldots, md$, and a sorted-copy check certifies $U^{\uparrow} \sim U$ and $a_z \le a_{z+1}$. Each address's contributions are now consecutive: the three contributions to $(0, 0)$ form one run.

**Phase 4: Computing the patch.** A group-by check certifies each run's total: with the run-end flags $f_z = \mathbf{1}[a_z \neq a_{z+1}]$ and $f_{md} = 1$, the running sums $c_1 = v_1^{\uparrow}$ and $c_z = v_z^{\uparrow} + (1 - f_{z-1})\, c_{z-1}$ restart at every boundary, and the flagged rows form the patch

$$P = \{(a_z,\ c_z) :\ f_z = 1\}, \qquad \Delta_{a_z} = c_z,$$

one entry per touched address; the other rows become dummies, so $P$ keeps the height $md$ and hides how many counters changed.

In the example, the run at $(0,0)$ sums to 340 with running sums 100, 300, 340; a flag inside the run or any other total fails a check.

**Phase 5: Installing the patch.** It remains to check that the new array applies the patch and preserves every other counter; checking only the claimed new values would leave the rest of the array unconstrained. For every patch entry $(a, \Delta_a)$, the prover supplies the read value $r_a$, and the circuit forms the reads and the writes

$$R = \{(a,\ r_a)\}_{(a,\Delta_a)\in P}, \qquad R^+ = \{(a,\ r_a + \Delta_a)\}_{(a,\Delta_a)\in P},$$

each of height $md$. Pairing each counter with its address, $\mathcal{M}_{t-1} = \{(a, S_{t-1}[a])\}_{0\le a<dw}$ and $\mathcal{M}_t = \{(a, S_t[a])\}_{0\le a<dw}$, each of height $dw$, one same-rows check certifies the balance

$$\mathcal{M}_{t-1} \uplus R^+ = \mathcal{M}_t \uplus R, \tag{1}$$

where $\uplus$ is multiset union and a dummy patch row adds the same fixed element to both sides, so both sides have the public height $dw + md$. Because every element carries its address, the balance holds address by address: at a touched counter $a$,

$$\{S_{t-1}[a],\ r_a + \Delta_a\} = \{S_t[a],\ r_a\} \quad\Longrightarrow\quad S_t[a] = S_{t-1}[a] + \Delta_a,$$

and at an untouched counter, with no read or write to balance against, $S_t[a] = S_{t-1}[a]$. In the figure, the read at $(0,0)$ is forced to 60 and the new counter to 400: a claimed read of 90 matches neither the old 60 nor its own write 430, and changing the untouched $(0,3)$ from 0 to 7 fails because no patch entry covers it.

**Cost.** Each phase is a linear pass over its tables, so the circuit uses

$$\underbrace{\Theta(m+dw)}_{\text{1: bind}} + \underbrace{\Theta(md)}_{\text{2: derive}} + \underbrace{\Theta(md)}_{\text{3: group}} + \underbrace{\Theta(md)}_{\text{4: patch}} + \underbrace{\Theta(dw+md)}_{\text{5: install}} = \Theta(md+dw)$$

circuit rows per batch at fixed widths, or $\Theta(d + dw/m)$ per record. Once $m \ge w$, the amortized cost is $\Theta(d)$, against $\Theta(dw)$ for the per-access scan.

**Correctness and obliviousness.** The openings fix the inputs, and the in-circuit hashes fix every address $a_{p,i}$. Since $U^{\uparrow} \sim U$ and the flags and sums are forced, each patch entry is the exact total $\Delta_a = \sum_{(p,i):\, a_{p,i}=a} v_p$ of its address, and the install balance turns these totals into $S_t[a] = S_{t-1}[a] + \Delta_a$ at every touched counter and $S_t[a] = S_{t-1}[a]$ elsewhere, which is the target. Every table has a public height ($m$, $md$, or $dw$), so collisions and the set of touched counters change only values and flags, never the circuit's structure. See the detailed proof in Appendix B.1.1.

*7.2.3 Extensions to Other Array Sketches.* Count-Min has supplied three components: deriving addressed contributions, combining them by address, and installing the patch. The first two carry over to any array sketch whose addresses depend only on the records and whose cells combine contributions by a sum or a maximum (for single bits, an OR); we replace addition with that aggregate. For a sum, combining is optional, since the install balance also holds over the raw contributions; our Count-Min circuit keeps it so that the same phases serve a maximum, where combining leaves one patch entry per cell. Installation needs one adaptation per sketch, which HyperLogLog illustrates; the packed Bloom filter circuit instead installs each access in turn.

**HyperLogLog.** HyperLogLog [28] keeps $s$ registers; a public hash sends key $k$ to register $h(k)$, a second assigns it a rank $\mu(k)$, and the update is $S_t[a] = \max(S_{t-1}[a], \mu_a)$ with $\mu_a = \max_{p:\, h(k_p)=a} \mu(k_p)$. The contributions $(h(k_p), \mu(k_p))$ and their grouping follow Count-Min with max in place of the sum, and the flagged rows form the patch $P = \{(a, \mu_a)\}$, one entry per touched register. For installation, a balance of values alone is not enough: with maximum, a read could be matched against its own write and leave the register unchanged. Each element therefore also carries a before/after tag, the time coordinate of memory checking [13, 60]: old cells and reads carry 0, writes carry 1, and each new cell a checked bit $\chi_a$, so that the balance becomes

$$\{(a, S_{t-1}[a], 0)\}_{0\le a<s} \uplus \{(a, \max(r_a, \mu_a), 1)\}_{(a,\mu_a)\in P} = \{(a, S_t[a], \chi_a)\}_{0\le a<s} \uplus \{(a, r_a, 0)\}_{(a,\mu_a)\in P},$$

which again forces the old read and the updated maximum at every patched register. The circuit uses $\Theta(m+s)$ rows, or $\Theta(1)$ per record once $m \ge s$ (Appendix B.2).

**Bloom filter.** The partitioned variant of the Bloom filter [12] has $k$ slices of $W$ bits (the slice count $k$ is unrelated to a record's key $k_p$), and a key sets one bit per slice: $S_t[j,b] = S_{t-1}[j,b] \vee \mathbf{1}[\exists \text{ real } p : h_j(k_p) = b]$ for slice $j$ and bit $b$, so contributions combine by OR and a dummy record sets no bit. Our deployed circuit packs eight bits per byte and processes the batch's $mk$ (record, slice) accesses in a fixed public order, without combining them by address: each access carries a timestamp, a finer version of HyperLogLog's before/after tag, so repeated accesses to one byte are matched in access order, and the circuit splits the accessed byte at the addressed bit so that the access sets exactly that bit. This costs $\Theta(mk + kW/8)$ circuit rows, or $\Theta(k)$ per record once $m \ge W/8$, with $mk < 2^{16}$ (Appendix B.3). The evaluation proves Bloom batches in leaves of at most 1 024 records, within this bound; a batch of the default size $m = 2^{14}$ exceeds it and runs an earlier circuit that stores one bit per cell.

## 7.3 Pattern B: Reducing a Bounded Summary

*7.3.1 Misra–Gries and the Key Idea.* For this kind of pattern, we require a public capacity $s$ for the summary and pad unused slots with dummy entries. Together with a fixed schedule of checks, this padding makes the circuit oblivious: its size and structure do not reveal how many entries the private data causes the sketch to retain. To guarantee that every update fits, $s$ must cover the sketch's worst-case size over the supported stream range. The Misra–Gries heavy-hitter summary [47] stores $s$ keys with approximate counts. A batch first merges its records into the stored entries, which can produce more than $s$ candidates, and the sketch's rule then *reduces* the candidates back to $s$. We certify the batch merge rule of Agarwal et al. [1], whose output undercounts each kept key by at most the total weight divided by $s+1$.

**Target.** With the batch $B_t$ of Section 7.2, write $S_t(k)$ for the count that $S_t$ stores for key $k$, or zero if $k$ is absent. The rule merges each key's old count with its batch amounts, reads a *cutoff*, the $(s{+}1)$-st largest merged count, subtracts it from every merged count, and keeps the keys whose counts stay positive, at most $s$ of them.

**Misra–Gries update circuit.**
**Input.** Public label $\mathrm{pub}_t$ and capacities $m, s$; private batch $B_t$, old and new summaries $S_{t-1}, S_t$, their commitment openings, and intermediate tables.

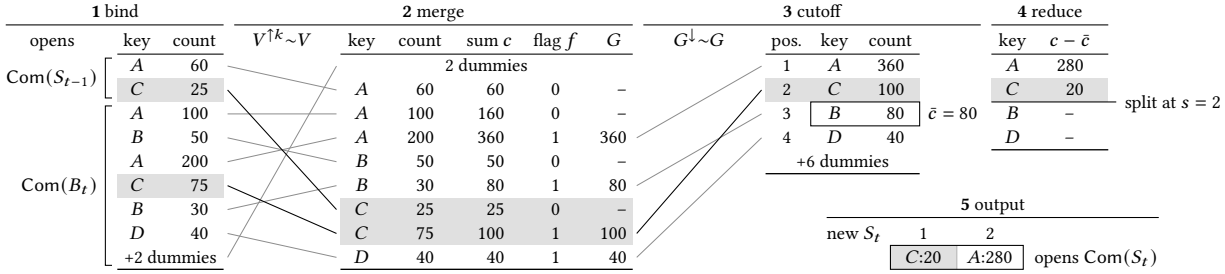



**Figure 4: The five phases of a Misra–Gries update on the running batch ($s = 2$, old summary $\{A{:}60,\ C{:}25\}$).**

**Output.** A proof $\pi_t$ certifying the new state commitment; the summaries and intermediate tables remain private.
**Verification target.** The batch and summaries open to their commitments. With $c(k) = S_{t-1}(k) + \sum_{p:\, k_p = k} v_p$ for every key $k$ of an old entry or batch record (an absent old key counting zero), and $\bar{c}$ the $(s{+}1)$-st largest of these counts, or zero if there are at most $s$ keys,

$$S_t = \{(k,\ c(k) - \bar{c}) :\ c(k) > \bar{c}\},$$

stored in $s$ slots, the unused slots holding a fixed dummy entry.

**Challenge.** Merging is the easy half: as for a Count-Min counter, a key's merged count depends only on its own old count and the batch. The difficulty is the reduction. Which keys survive, and by how much every count shrinks, depends on the cutoff $\bar{c}$, an order statistic of *all* merged counts, so checking the $s$ entries of the new summary cannot establish that $\bar{c}$ was read correctly. A direct circuit replays the per-record rule instead: each record searches the $s$ stored keys for its own, a private access, and a miss on a full summary decrements every entry, $\Theta(s)$ rows per record.
**Key idea.** Both halves become linear passes once the candidates are in the right order. In key order, a key's old entry and its batch records are consecutive, so one group-by pass merges them, exactly as Count-Min merged contributions by address. In decreasing count order, the cutoff is simply the count at the public position $s + 1$, and the survivors are the rows before it. We therefore certify the update through *two checked orders*: the prover supplies both sorted copies, the circuit checks each against its source, and every decision, from the merged counts to the cutoff and the survivors, is read from a checked table rather than chosen by the prover.

*7.3.2 The Five Phases.* **Running example.** We continue with $s = 2$ slots, the old summary $\{A{:}60,\ C{:}25\}$, and the running batch of 6 records and 2 dummies; Figure 4 traces the phases.
**Phase 1: Binding the batch and summaries.** As for Count-Min, the circuit opens $B_t$, $S_{t-1}$, and $S_t$ against $\mathrm{pub}_t$. It then forms the *candidate table*

$$V = \{(k,\ S_{t-1}(k))\}_{k \text{ stored in } S_{t-1}} \ \uplus\ \{(k_p,\ v_p)\}_{1 \le p \le m},$$

with unused slots and the batch's dummies as dummy rows of key 0 and count 0, so $V$ has the public height $n = s + m$. In the example, $n = 10$: two old entries, six records, and two dummies.
**Phase 2: Merging counts by key.** The prover supplies the copy $V^{\uparrow k}$ of $V$ sorted by key, dummies first, with rows $(k_z, v_z^{\uparrow})$; a sorted-copy check certifies $V^{\uparrow k} \sim V$ and $k_z \le k_{z+1}$, so each key's rows are consecutive. A group-by check with flags $f_z = \mathbf{1}[k_z \ne k_{z+1}]$ then sums each run, $c_z = v_z^{\uparrow} + (1 - f_{z-1})\, c_{z-1}$, and the flagged row of key $k$ carries its merged count

$$c(k) = S_{t-1}(k) + \sum_{p:\, k_p = k} v_p;$$

the other rows become dummies, so the resulting table $G$ keeps the height $n$. In the example,

$$V^{\uparrow k} :\ A{:}60,\ A{:}100,\ A{:}200,\ B{:}50,\ B{:}30,\ C{:}25,\ C{:}75,\ D{:}40;$$
$$G \text{ (real rows)} :\ A{:}360,\ B{:}80,\ C{:}100,\ D{:}40.$$

Both checks are needed. A prover might list $C$'s old entry and its batch amount as separate candidates, $C{:}25$ and $C{:}75$; the cutoff would fall to 75, and the summary $\{A{:}285,\ B{:}5\}$, two well-formed entries, would evict $C$. Listing the two rows apart violates the sorted order, and a run boundary between them when they are adjacent violates the flag check.
**Phase 3: Reading the cutoff.** The prover supplies a copy $G^{\downarrow} = ((k_1, c_1), \ldots, (k_n, c_n))$ of $G$ sorted by decreasing count, real rows before dummies and ties broken by key, and a sorted-copy check certifies $G^{\downarrow} \sim G$ and $c_1 \ge c_2 \ge \cdots \ge c_n$. The $(s{+}1)$-st largest count now sits at the public position $s + 1$, so the circuit reads $\bar{c} = c_{s+1}$ there, or zero if that row is a dummy, that is, if at most $s$ distinct keys remain. In the example, $G^{\downarrow}$ is $A{:}360,\ C{:}100,\ B{:}80,\ D{:}40$, and position 3 gives $\bar{c} = 80$. A prover claiming the cutoff 100, which would leave only $\{A{:}260\}$, fails because position 3 holds $B{:}80$, and any order that moved $C{:}100$ there fails the sorted-copy check.
**Phase 4: Reducing the counts.** For each row $j \le s$ of $G^{\downarrow}$, the circuit range-checks $c_j - \bar{c} \ge 0$, as the checked order implies, and forms the reduced row

$$\tilde{g}_j = \begin{cases} (k_j,\ c_j - \bar{c}) & c_j > \bar{c}, \\ \text{dummy} & c_j = \bar{c}, \end{cases} \qquad j = 1, \ldots, s,$$

since the rule keeps only counts strictly above $\bar{c}$. The rows after position $s$ need no arithmetic: $c_j \le \bar{c}$ for $j > s$, so none survives. In the example, $A{:}360$ and $C{:}100$ become $A{:}280$ and $C{:}20$, while $B$, at the cutoff, and $D$, below it, are dropped.
**Phase 5: Binding the output.** Because the count order already places the survivors in the first $s$ rows of $G^{\downarrow}$, no further permutation is needed: the circuit binds $\tilde{g}_1, \ldots, \tilde{g}_s$, dummies included, slot by slot to the opened $S_t$, in the stored order of Appendix B.4. We call this cut at a public row a *fixed split*. Since every slot is bound, no survivor can be dropped and no entry invented. In the example, $S_t$ stores $(C{:}20,\ A{:}280)$ in slot order.

**Cost.** Every table has height $n = s + m$ or less, so the circuit uses

$$\underbrace{\Theta(n)}_{\text{1: bind}} + \underbrace{\Theta(n)}_{\text{2: merge}} + \underbrace{\Theta(n)}_{\text{3: cutoff}} + \underbrace{\Theta(s)}_{\text{4: reduce}} + \underbrace{\Theta(s)}_{\text{5: output}} = \Theta(s+m)$$

rows per batch at fixed widths, or $\Theta(1 + s/m)$ per record. Once $m \geq s$, the amortized cost is $\Theta(1)$ rather than the replay's $\Theta(s)$.

**Correctness and obliviousness.** The openings fix the batch and both summaries. The key-sorted copy and group-by make each merged count the exact sum of its key's rows, so no key can be split. The count-sorted copy fixes the order, so $\bar{c} = c_{s+1}$ is the true cutoff. And the range-checked differences and the fixed split force $S_t = \{(k,\ c(k) - \bar{c}) :\ c(k) > \bar{c}\}$, the target. Every table has a public height ($m$, $s$, or $n$), so the number of candidates, the cutoff, and the survivors change only values and flags, never the circuit's structure. See the detailed proof in Appendix B.4.

*7.3.3 Extensions to Other Bounded Summaries.* Misra–Gries has supplied three components: candidates made complete by a checked order, a reduction decision read from a checked table, and survivors bound to the output by a fixed split. A new bounded summary supplies its own candidate representation, order, and reduction rule. KMV changes the decision and the placement; Greenwald–Khanna applies its decision repeatedly, and its threshold depends on the record count, which must be tied to the committed history.

**KMV.** For distinct counting, KMV [2, 5, 20] keeps the $s$ smallest distinct values of a public hash $h$ of the keys: $S_t = \text{bottom}_s\big(S_{t-1} \cup h(B_t)\big)$. The candidates are the old values and the hashes of the batch keys, sorted ascending by a sorted-copy check into $u_1 \leq \cdots \leq u_n$. The decision needs no order statistic, only the public capacity: adjacent comparisons fix the first-occurrence flags $d_i = \mathbf{1}[i = 1 \text{ or } u_i \neq u_{i-1}]$ ($d_i = 0$ on dummies), a prefix count $\sigma_i = \sum_{j \leq i} d_j$ gives each value's rank among the distinct values, and a value is kept when $d_i = 1$ and $\sigma_i \leq s$. Since the kept values need not sit in the first $s$ rows, a fixed split no longer works; instead, each kept value's rank is its output position, and a same-rows check on (value, position) pairs, padded with dummies on both sides, binds the state. We call this placement by computed positions *stable compaction*; Greenwald–Khanna reuses it. The cost is $\Theta(m + s)$ rows per batch, or $\Theta(1)$ per record once $m \geq s$, and the rule reproduces sequential insertion. Appendix B.5 gives the construction.

**Greenwald–Khanna.** The GK sketch [32] estimates the *rank* of a value, the number of records no larger than it, within $\varepsilon N_t$ after $N_t$ records, for a public accuracy parameter $\varepsilon \in (0, 1)$. We certify a batch rule built on the two-sided variant of Gribelyuk et al. [33]; its summary keeps at most $s'$ tuples $(e, g_1, g_2, \Delta_1, \Delta_2)$ in value order: value $e$ covers $g_1 + g_2 + 1$ records, and $\Delta_1, \Delta_2$ bound its rank error, with the invariant $g_1 + \Delta_1 \leq \theta_t$ and $g_2 + \Delta_2 \leq \theta_t$ for the threshold $\theta_t = \lfloor \varepsilon N_t \rfloor$. The candidates are the old tuples and one new tuple per real batch record, sorted by value with a sorted-copy check as in Misra–Gries; each new tuple takes its bounds $\Delta_1, \Delta_2$ from its nearest old neighbors (Appendix B.6). The decision is *fusion*: a tuple absorbs a newer neighbor's coverage into one side when that side's $g + \Delta$ stays at most $\theta_t$, and, as with Misra–Gries's cutoff, the circuit computes each decision from checked fields.

Two things are new. First, a round fuses only disjoint pairs, so fusion repeats for $L_f = \lceil \log_2(s' + m) \rceil$ rounds; KMV's stable compaction closes the gaps after each round without sorting again, and a fixed split binds the final survivors to $S_t$. Second, a prover could overstate the record count to raise $\theta_t$; since fusion moves coverage between tuples without creating any, the circuit checks that $\sum_{S_t}(g_1 + g_2 + 1) = N_{t-1} + n_t$ for the batch's $n_t$ real records, which fixes $N_t$ and hence $\theta_t$. The prior count $N_{t-1}$ and threshold $\theta_t$ stay private: the circuit binds them, so the batch proof still exposes only $\text{pub}_t$. The cost is $\Theta((s' + m)L_f)$ rows per batch, or $\Theta(\log s')$ per record at $m = \Theta(s')$.

Appendix B shows all six update circuits, with their correctness and obliviousness, and gives the query circuits. Table 1 summarizes the pattern of each sketch, supported query, and amortized circuit cost per update.

**Table 1: Sketch patterns, queries, and amortized update costs.**

| Family | Pattern | Amortized per update | Query |
|---|---|---|---|
| Count-Min | A | $\Theta(d)$ | Frequency or total weight of a queried key |
| HyperLogLog | A | $\Theta(1)$ | Number of distinct keys |
| Bloom | A | $\Theta(k)$ | Key membership (false positives possible) |
| KMV | B | $\Theta(1)$ | Number of distinct keys |
| MG (SS) | B | $\Theta(1)$ | Sum of the $K$ largest stored counts |
| GK | B | $\Theta(\log s')$ | Rank of a queried value |

## 7.4 Query Circuits

The update circuits certify the sketch state reached by the logged batches of a stream prefix. To certify an answer, a query circuit opens this state against $\text{Com}(S_t)$ and checks the declared query rule of Table 1. Its public statement is the state commitment, the query, and the reported answer, and the verifier matches the commitment to the one certified by the history proof or the live peaks. The circuit can rely on the state invariants that the update circuits established, and it uses the building blocks of Section 7.1 on the opened state.

A Count-Min point query hashes the requested key $k$ to its counters and certifies

$$res \leq S_t[i, h_i(k)] \ \text{ for all } i, \qquad \prod_i \big(res - S_t[i, h_i(k)]\big) = 0,$$

so that the answer is their minimum. A Misra–Gries query releases $\sum_{j \leq K} c_{(j)}$ for the stored counts in decreasing order $c_{(1)} \geq c_{(2)} \geq \cdots$, with adjacent comparisons confirming their order; the keys are distinct because the update circuit stores each key at most once. A GK rank query finds the requested value's position among the stored values, derives rank bounds $r_{\min}$ and $r_{\max}$ from the tuples' coverage counts and error bounds, and certifies $2\,res + rem = r_{\min} + r_{\max}$ with a bit $rem$ and a range-checked $res$, so that $res = \lfloor (r_{\min} + r_{\max})/2 \rfloor$. The HyperLogLog query certifies the raw estimate without the small-range correction, so its answer is reliable only once it exceeds $5s/2$ for $s$ registers. At fixed sketch capacities, this work does not grow with the stream, although opening the whole state costs $\Theta(dw)$ for Count-Min. Appendix B gives the detailed checks and the other families' query circuits.

# 8 EVALUATION

We evaluate zkSketch across six sketch families to assess its efficiency relative to zk-Analytics under a common memory budget.

The evaluation examines how query frequency and batch size affect performance, which phases dominate proving time, and how proving scales with stream length and the number of machines.

## 8.1 Setup

**Implementation.** We implement all six sketches' update and query circuits based on Plonky3 [55] over BabyBear, using Poseidon2[31] for hashing. And zkSketch targets a security level of 123 bits. Every reported proof is verified end to end.

**Hardware and data.** Absolute timings use an AMD Threadripper PRO 9975WX with each system pinned to 32 physical cores. Scalability and dataset ratios come from isolated 32-core processes on an EPYC 9555, following the hardware-islands methodology [56], and are applied to the current timings. We use Uniform and Gaussian synthetic streams, AOL clicks [54], and Netflix-Prize ratings [49]; AOL is the default.

**Parameter setting.** Count-Min uses $d = 3$ and $w = 2^{14}$; HyperLogLog $s = 2^{14}$ registers; Misra–Gries $s = 2^{14}$ summary slots; Greenwald–Khanna a declared tuple capacity $s' = 2^{11}$ with accuracy parameter $1/\varepsilon = 51$ over a 22-bit value domain; the Bloom filter $k = 7$ slices of $W = 2^{14}$ bits; KMV the $s = 2^{14}$ smallest distinct hash values. Unless otherwise stated, the stream is $N = 2^{20}$ records processed by the proof forest in $t = 64$ batches of $m = 2^{14}$.

**Comparison.** We compare zkSketch with zk-Analytics [74], which, like zkSketch, proves queries on streaming data in zero knowledge. It proves each epoch of records by re-executing the sketch's update program in a zkVM, and each query by a second program that merges the epoch summaries. Its code base implements only Count-Min, but the zkVM proves the execution of any Rust program, so each of our other sketches needs only a new update and merge program in its otherwise unchanged pipeline. We write these as plain textbook code for HyperLogLog, SpaceSaving, the Bloom filter, and KMV, and for Greenwald–Khanna as the plain-code form of our batch rule. zk-Analytics gives 100 bits of security, whereas ours give 123. In addition, zkSketch uses Poseidon2 [31], a hash designed for 128-bit security, for in-circuit key hashing and state commitments, whereas zk-Analytics hashes keys with FNV-1a [50] and splitmix64 [61], which are fast but not cryptographic: their 64-bit output caps collision resistance at 32 bits.

**Memory budget.** Larger leaves amortize state checks and avoid compositions, but require more memory. RISC Zero's $2^{20}$-cycle segments keep zk-Analytics' measured peak near 9.3 GiB. We keep zkSketch below this peak by limiting each leaf to $L = 1024$ records for Count-Min, HyperLogLog, and Bloom, 4096 for Misra–Gries and KMV, and 512 for GK. Larger batches are split into leaves and composed. The complete prover peaks at 4.8–7.3 GiB, including the coordinator and queries. We apply this budget to the whole-stream comparison and the batch-size sweep in Figure 5. The phase breakdown and the earlier runs supplying scalability ratios use larger leaves.

**Metrics.** We report proving time, verification time, proof size, and peak resident memory.

## 8.2 Comparison with zk-Analytics

**Proving time and memory.** zkSketch obtains 23–82× faster whole-stream proving while using 21–48% less peak memory than zk-Analytics (Table 2). The largest gains are for KMV and Misra–Gries, where we check one selection or reduction over the batch instead of proving a sequence of individual updates. Their 4096-record leaves also spread state checks and composition costs over more records. GK requires repeated rounds of tuple fusion and compaction, making it the slowest sketch to prove. Its smaller 512-record leaves nevertheless keep peak memory at 5.0 GiB, below Misra–Gries's 7.3 GiB. Thus, total proving work and peak memory need not grow together.

**Table 2: Whole-stream comparison at $N = 2^{20}$.**

| | | CM | HLL | MG | GK | Bloom | KMV |
|---|---|---|---|---|---|---|---|
| prove (hour) | zk-Analytics | 71.3 | 66.7 | 79.0 | 193 | 71.4 | 69.2 |
| | zkSketch | 2.84 | 2.94 | 1.05 | 7.10 | 2.88 | 0.84 |
| | speedup | **25×** | **23×** | **75×** | **27×** | **25×** | **82×** |
| verify (ms) | zk-Analytics | 623 | 623 | 626 | 4 975 | 622 | 623 |
| | zkSketch | 15.0 | 15.0 | 15.1 | 14.9 | 15.0 | 15.0 |
| proof size (MiB) | zk-Analytics | 13.8 | 13.7 | 13.7 | 110 | 13.7 | 13.7 |
| | zkSketch | 0.75 | 0.75 | 0.75 | 0.75 | 0.75 | 0.75 |
| peak memory (GiB) | zk-Analytics | 9.2 | 9.2 | 9.2 | 9.2 | 9.3 | 9.2 |
| | zkSketch | **5.0** | **6.8** | **7.3** | **5.0** | **4.8** | **5.0** |

**Table 3: Proving time (ms/update), including queries every $Q$ positions. zkS: zkSketch; zkA: zk-Analytics; ratio: zkA/zkS.**

| | $Q = 10^3$ ($n_q = 131$) | | | $Q = 10^4$ ($n_q = 13$) | | | $Q = 10^5$ ($n_q = 1$) | | |
|---|---|---|---|---|---|---|---|---|---|
| | zkS | zkA | ratio | zkS | zkA | ratio | zkS | zkA | ratio |
| CM | 10.2 | 2 758 | 271× | 10.0 | 274 | 27× | 9.7 | 245 | 25× |
| HLL | 18.1 | 973 | 54× | 10.7 | 239 | 22× | 9.8 | 229 | 23× |
| MG | 14.1 | 18 881 | 1 340× | 4.1 | 459 | 112× | 3.3 | 270 | 81× |
| GK | 22.5 | 751 | 33× | 21.8 | 666 | 31× | 21.2 | 664 | 31× |
| Bloom | 10.8 | 605 | 56× | 10.1 | 251 | 25× | 10.1 | 244 | 24× |
| KMV | 11.4 | 779 | 69× | 3.3 | 247 | 74× | 2.7 | 231 | 85× |

**Verification and proof size.** Across all six sketches, this proof occupies 0.75 MiB and verifies in 14.9–15.1 ms, about 42× faster than zk-Analytics for five sketches and 334× for GK. The larger GK advantage comes from the number of proofs the baseline must check: its smaller epochs produce 512 receipts, compared with 64 for the other sketches. This makes its total verification time and proof size roughly eight times larger.

**Query frequency.** We vary the interval $Q$ between queries in a $2^{17}$-position stream to examine the combined proving cost of updates and queries (Table 3). To answer at the requested stream position, we may need to close a leaf before it fills. This particularly affects Misra–Gries and KMV: at $Q = 10^3$, their 4096-slot leaves close after about 1000 records, leaving roughly three quarters as padding. Their cost consequently rises to about 4.2× that at $Q = 10^5$. HyperLogLog is sensitive for a different reason: answer proofs account for about 45% of its time at $Q = 10^3$. Count-Min, GK, and Bloom vary by only 5–7%.

Frequent queries are also costly for zk-Analytics because each query merges the epoch summaries accumulated so far. Our query circuits instead use the current certified sketch. For Misra–Gries, the baseline's cost rises from 270 to 18 881 ms/update as $Q$ falls from $10^5$ to $10^3$. Despite the additional padding in our leaves, the advantage over zk-Analytics therefore grows from 81× to 1340×.

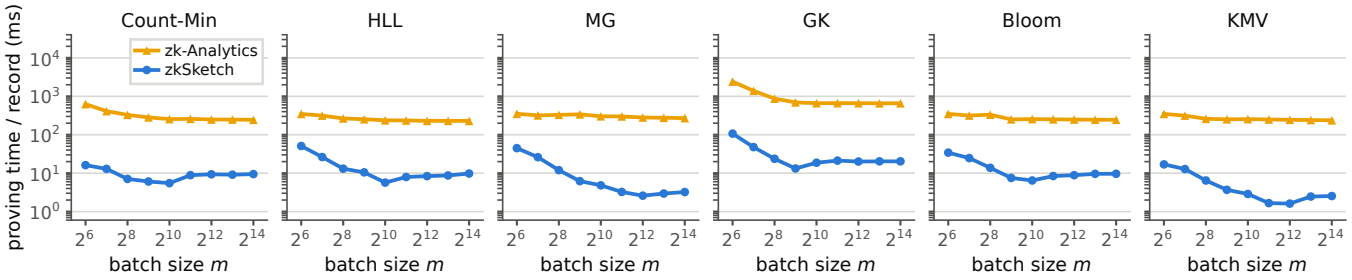



**Figure 5: Proving time per record under the comparison memory budget.**

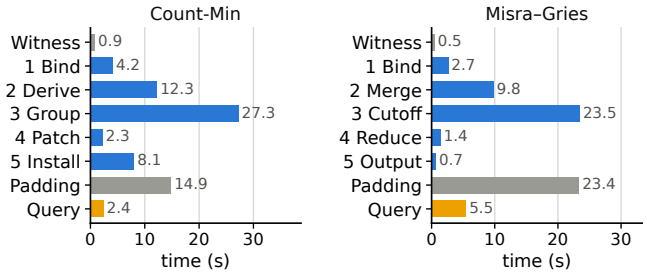



**Figure 6: Proving time of one Count-Min leaf and one Misra–Gries leaf by phase.**

**Batch size.** zkSketch obtains the lowest cost per record when a batch approximately fills one leaf (Figure 5). The memory budget explains this pattern. Below the leaf limit $L$, increasing the batch size spreads state opening and fixed proof work over more records, reducing the cost from 16–107 ms/record at $m = 2^6$ to 1.6–13 ms near $L$. Above $L$, we must split the batch into $m/L$ leaves and add $m/L-1$ compositions. Each composition costs 4.2–7.1 s, comparable to a leaf proof, so the cost per record rises toward a plateau. zk-Analytics also benefits from larger batches, but still pays to prove the program's instructions and memory accesses. Its cost levels off near 230–340 ms/record (660–700 for GK). Our speedup peaks at 40–151× near $L$ and falls to 23–93× at $m = 2^{14}$ as compositions add work. Whole-stream speedups are slightly lower because they also include compositions across epochs.

## 8.3 Proving Cost by Phase

Checking order accounts for much more proving work than the update arithmetic itself (Figure 6). To examine this cost, we apportion each leaf's joint proof time among tables by their committed cells, then among phases by their contributed rows; we time witness generation and queries separately. Count-Min grouping accounts for 27.3 s (40%), compared with 2.3 s for computing the patch. Similarly, Misra–Gries cutoff selection accounts for 23.5 s (38%), compared with 2.1 s for reduction and output together. Although we sort outside the circuit, the proof must still establish that the sorted copy contains the same entries in the required order. Trace padding adds another substantial cost. Rounding table heights to powers of two accounts for 22% of Count-Min's time and 38% of Misra–Gries's time, nearly as much as cutoff selection in the latter.

**Table 4: Proving time (s) for varying stream lengths $N$ on one machine and varying machine counts $M$ at $N = 2^{20}$.**

| | CM | HLL | MG | GK | Bloom | KMV |
|---|---|---|---|---|---|---|
| $N = 2^{16}$ | 635 | 657 | 231 | 1 591 | 645 | 184 |
| $N = 2^{17}$ | 1 275 | 1 319 | 468 | 3 188 | 1 294 | 375 |
| $N = 2^{18}$ | 2 555 | 2 644 | 943 | 6 383 | 2 592 | 756 |
| $N = 2^{19}$ | 5 116 | 5 294 | 1 892 | 12 773 | 5 188 | 1 518 |
| $N = 2^{20}$ | 10 237 | 10 592 | 3 791 | 25 552 | 10 380 | 3 041 |
| $M = 2$ | 5 241 | 5 374 | 1 926 | 13 018 | 5 275 | 1 549 |
| | (1.95×) | (1.97×) | (1.97×) | (1.96×) | (1.97×) | (1.96×) |
| $M = 3$ | 3 663 | 3 730 | 1 346 | 9 121 | 3 670 | 1 081 |
| | (2.79×) | (2.84×) | (2.82×) | (2.80×) | (2.83×) | (2.81×) |
| $M = 4$ | 2 713 | 2 755 | 994 | 6 752 | 2 714 | 799 |
| | (3.77×) | (3.84×) | (3.81×) | (3.78×) | (3.82×) | (3.81×) |

## 8.4 Scalability

**Stream length.** Table 4 shows that the proving time grow linearly with $N$, keeping the cost per record nearly constant. Across $N = 2^{16}$ to $2^{20}$, a 16× increase in stream length raises proving time by 16.1–16.5×, with less than 4% variation in per-record.

**Machines.** The proof forest lets machines prove disjoint stream segments and perform most compositions locally. This parallel work yields speedups of 1.95–1.97×, 2.79–2.84×, and 3.77–3.84× on two, three, and four machines, respectively (Table 4). The final cross-machine compositions must wait for the local proofs, limiting the gains; four machines achieve 94–96% parallel efficiency.

**Datasets.** Changing the input distribution has little effect on proving time. The six families vary by only 0.5–1.1% across datasets. This agrees with the circuit design, whose fixed table sizes make different inputs change the witness values rather than the number of rows checked. Appendix C gives the per-dataset results.

# 9 CONCLUSION

zkSketch certifies sketch updates and queries over committed stream prefixes while hiding records and sketch states. Batch-effect certificates reduce update-proving work, and a proof forest compresses the history while allowing parallel proving. Under zk-Analytics' memory budget, the whole-stream comparison gives speedups of 23–82× across six sketches; the 0.75 MiB history proof verifies in about 15 ms independently of stream length.

# REFERENCES


[1] Pankaj K. Agarwal, Graham Cormode, Zengfeng Huang, Jeff M. Phillips, Zhewei Wei, and Ke Yi. 2012. Mergeable Summaries. In *Proceedings of the 31st ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS)*. 23–34. https://doi.org/10.1145/2213556.2213562

[2] Apache Software Foundation. 2026. Apache DataSketches. https://datasketches.apache.org/ Accessed 2026-05-03.

[3] Arvind Arasu, Ken Eguro, Raghav Kaushik, Donald Kossmann, Pingfan Meng, Vineet Pandey, and Ravi Ramamurthy. 2017. Concerto: A High Concurrency Key-Value Store with Integrity. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD)*. ACM, New York, NY, USA, 251–266. https://doi.org/10.1145/3035918.3064030

[4] Kassem Bagher and Shangqi Lai. 2023. SGX-Stream: A Secure Stream Analytics Framework in SGX-enabled Edge Cloud. *Journal of Information Security and Applications* 72 (2023), 103403. https://doi.org/10.1016/j.jisa.2022.103403

[5] Ziv Bar-Yossef, T. S. Jayram, Ravi Kumar, D. Sivakumar, and Luca Trevisan. 2002. Counting Distinct Elements in a Data Stream. In *Randomization and Approximation Techniques in Computer Science (RANDOM 2002) (Lecture Notes in Computer Science)*, Vol. 2483. Springer, 1–10. https://doi.org/10.1007/3-540-45726-7_1

[6] Johes Bater, Gregory Elliott, Craig Eggen, Satyender Goel, Abel Kho, and Jennie Rogers. 2017. SMCQL: Secure Querying for Federated Databases. *Proceedings of the VLDB Endowment* 10, 6 (2017), 673–684. https://doi.org/10.14778/3055330.3055334

[7] Carsten Baum, Ivan Damgård, and Claudio Orlandi. 2014. Publicly Auditable Secure Multi-Party Computation. In *Security and Cryptography for Networks (SCN 2014) (Lecture Notes in Computer Science)*, Vol. 8642. Springer, 175–196. https://doi.org/10.1007/978-3-319-10879-7_11

[8] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. 2018. Fast Reed-Solomon Interactive Oracle Proofs of Proximity. In *45th International Colloquium on Automata, Languages, and Programming (ICALP)*.

[9] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. 2018. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Paper 2018/046. https://eprint.iacr.org/2018/046

[10] Jon Louis Bentley and James B. Saxe. 1980. Decomposable Searching Problems I: Static-to-Dynamic Transformation. *Journal of Algorithms* 1, 4 (1980), 301–358.

[11] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. 2013. Recursive Composition and Bootstrapping for SNARKs and Proof-Carrying Data. In *ACM Symposium on Theory of Computing (STOC)*. 111–120.

[12] Burton H. Bloom. 1970. Space/Time Trade-offs in Hash Coding with Allowable Errors. *Commun. ACM* 13, 7 (1970), 422–426. https://doi.org/10.1145/362686.362692

[13] Manuel Blum, William S. Evans, Peter Gemmell, Sampath Kannan, and Moni Naor. 1994. Checking the Correctness of Memories. *Algorithmica* 12, 2/3 (1994), 225–244. https://doi.org/10.1007/BF01185212 Preliminary version in FOCS 1991.

[14] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. 2017. Practical Secure Aggregation for Privacy-Preserving Machine Learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 1175–1191. https://doi.org/10.1145/3133956.3133982

[15] Dan Boneh, Elette Boyle, Henry Corrigan-Gibbs, Niv Gilboa, and Yuval Ishai. 2021. Lightweight Techniques for Private Heavy Hitters. In *2021 IEEE Symposium on Security and Privacy (S&P)*. 762–776.

[16] Lukas Burkhalter, Anwar Hithnawi, Alexander Viand, Hossein Shafagh, and Sylvia Ratnasamy. 2020. TimeCrypt: Encrypted Data Stream Processing at Scale with Cryptographic Access Control. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. 835–850.

[17] T.-H. Hubert Chan, Elaine Shi, and Dawn Song. 2010. Private and Continual Release of Statistics. In *Automata, Languages and Programming, 37th International Colloquium (ICALP 2010), Part II (Lecture Notes in Computer Science)*, Vol. 6199. Springer, 405–417. https://doi.org/10.1007/978-3-642-14162-1_34

[18] Bing-Jyue Chen, Suppakit Waiwitlikhit, Ion Stoica, and Daniel Kang. 2024. ZKML: An Optimizing System for ML Inference in Zero-Knowledge Proofs. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys)*. 560–574. https://doi.org/10.1145/3627703.3650088

[19] Yan Chen, Ashwin Machanavajjhala, Michael Hay, and Gerome Miklau. 2017. PeGaSus: Data-Adaptive Differentially Private Stream Processing. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 1375–1388. https://doi.org/10.1145/3133956.3134102

[20] Edith Cohen and Haim Kaplan. 2007. Summarizing Data Using Bottom-k Sketches. In *Proceedings of the 26th Annual ACM Symposium on Principles of Distributed Computing (PODC)*. 225–234. https://doi.org/10.1145/1281100.1281133

[21] Graham Cormode and S. Muthukrishnan. 2005. An Improved Data Stream Summary: The Count-Min Sketch and Its Applications. *Journal of Algorithms* 55, 1 (2005), 58–75. https://doi.org/10.1016/j.jalgor.2003.12.001

[22] Henry Corrigan-Gibbs and Dan Boneh. 2017. Prio: Private, Robust, and Scalable Computation of Aggregate Statistics. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 259–282.

[23] Scott A. Crosby and Dan S. Wallach. 2009. Efficient Data Structures For Tamper-Evident Logging. In *USENIX Security Symposium*. 317–334.

[24] Cynthia Dwork. 2006. Differential privacy. In *International colloquium on automata, languages, and programming*. Springer, 1–12.

[25] Cynthia Dwork, Moni Naor, Toniann Pitassi, and Guy N. Rothblum. 2010. Differential Privacy under Continual Observation. In *Proceedings of the 42nd ACM Symposium on Theory of Computing (STOC)*. 715–724. https://doi.org/10.1145/1806689.1806787

[26] Amos Fiat and Adi Shamir. 1986. How to prove yourself: Practical solutions to identification and signature problems. In *Conference on the theory and application of cryptographic techniques*. Springer, 186–194.

[27] Dario Fiore and Ida Tucker. 2022. Efficient Zero-Knowledge Proofs on Signed Data with Applications to Verifiable Computation on Data Streams. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 1067–1080.

[28] Philippe Flajolet, Eric Fusy, Olivier Gandouet, and Frederic Meunier. 2007. HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm. In *Analysis of Algorithms 2007*. 127–146. https://algo.inria.fr/flajolet/Publications/FlFuGaMe07.pdf

[29] Minos Garofalakis, Joseph M. Hellerstein, and Petros Maniatis. 2007. Proof Sketches: Verifiable In-Network Aggregation. In *Proceedings of the 23rd IEEE International Conference on Data Engineering (ICDE)*. 996–1005. https://doi.org/10.1109/ICDE.2007.368958

[30] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. 1985. The Knowledge Complexity of Interactive Proof-Systems. In *Proceedings of the 17th Annual ACM Symposium on Theory of Computing (STOC)*. 291–304.

[31] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. 2023. Poseidon2: A Faster Version of the Poseidon Hash Function. In *Progress in Cryptology – AFRICACRYPT 2023 (LNCS)*, Vol. 14064. 177–203. https://doi.org/10.1007/978-3-031-37679-5_8

[32] Michael Greenwald and Sanjeev Khanna. 2001. Space-Efficient Online Computation of Quantile Summaries. In *Proceedings of the 2001 ACM SIGMOD International Conference on Management of Data*. 58–66. https://doi.org/10.1145/375663.375670

[33] Elena Gribelyuk, Pachara Sawettamalya, Hongxun Wu, and Huacheng Yu. 2024. Simple & optimal quantile sketch: Combining greenwald-khanna with khanna-greenwald. *Proceedings of the ACM on Management of Data* 2, 2 (2024), 1–25.

[34] Binbin Gu, Juncheng Fang, and Faisal Nawab. 2025. PoneglyphDB: Efficient Non-Interactive Zero-Knowledge Proofs for Arbitrary SQL-Query Verification. *Proceedings of the ACM on Management of Data* 3, 1, Article 63 (2025). https://doi.org/10.1145/3709713

[35] Ulrich Haböck. 2022. Multivariate Lookups Based on Logarithmic Derivatives. IACR Cryptology ePrint Archive, Report 2022/1530. https://eprint.iacr.org/2022/1530

[36] Aurélien Havet, Rafael Pires, Pascal Felber, Marcelo Pasin, Romain Rouvoy, and Valerio Schiavoni. 2017. SecureStreams: A Reactive Middleware Framework for Secure Data Stream Processing. In *Proceedings of the 11th ACM International Conference on Distributed and Event-based Systems (DEBS)*. 124–133. https://doi.org/10.1145/3093742.3093927

[37] Intel Corporation. 2025. Discontinued: Intel SGX Attestation Service. https://www.intel.com/content/www/us/en/developer/archive/tools/sgx-attestation-service-utilizing-epid.html. Archived developer notice: "On April 2, 2025, Intel discontinued the Intel SGX Attestation Service Utilizing Intel EPID." Undated page; accessed 2026-10-20.

[38] Peter Kairouz, H. Brendan McMahan, et al. 2021. Advances and Open Problems in Federated Learning. *Foundations and Trends in Machine Learning* 14, 1–2 (2021), 1–210. https://doi.org/10.1561/2200000083

[39] Georgios Kellaris, Stavros Papadopoulos, Xiaokui Xiao, and Dimitris Papadias. 2014. Differentially Private Event Sequences over Infinite Streams. *Proceedings of the VLDB Endowment* 7, 12 (2014), 1155–1166. https://doi.org/10.14778/2732977.2732989

[40] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. 2022. Nova: Recursive Zero-Knowledge Arguments from Folding Schemes. In *Advances in Cryptology – CRYPTO 2022, Part IV (Lecture Notes in Computer Science)*. Springer, 359–388. https://doi.org/10.1007/978-3-031-15985-5_13

[41] Christian Janos Lebeda and Jakub Tětek. 2023. Better Differentially Private Approximate Histograms and Heavy Hitters using the Misra-Gries Sketch. In *Proceedings of the 42nd ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS)*. 79–88. https://doi.org/10.1145/3584372.3588673

[42] Feifei Li, Ke Yi, Marios Hadjieleftheriou, and George Kollios. 2007. Proof-Infused Streams: Enabling Authentication of Sliding Window Queries on Streams. In *Proceedings of the 33rd International Conference on Very Large Data Bases (VLDB)*. ACM, 147–158.

[43] Xiling Li, Chenkai Weng, Yongxin Xu, Xiao Wang, and Jennie Rogers. 2023. ZKSQL: Verifiable and Efficient Query Evaluation with Zero-Knowledge Proofs. *Proceedings of the VLDB Endowment* 16, 8 (2023), 1804–1816. https://doi.org/10.14778/3594512.3594513

[44] Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. 2011. Certifying Algorithms. *Computer Science Review* 5, 2 (2011), 119–161. https://doi.org/10.1016/j.cosrev.2010.09.009

[45] Ralph C. Merkle. 1988. A Digital Signature Based on a Conventional Encryption Function. In *Advances in Cryptology – CRYPTO '87 (Lecture Notes in Computer Science)*, Vol. 293. Springer, 369–378.

[46] Ahmed Metwally, Divyakant Agrawal, and Amr El Abbadi. 2006. An Integrated Efficient Solution for Computing Frequent and Top-k Elements in Data Streams. *ACM Transactions on Database Systems* 31, 3 (2006), 1095–1133. https://doi.org/10.1145/1166074.1166084

[47] Jayadev Misra and David Gries. 1982. Finding Repeated Elements. *Science of Computer Programming* 2, 2 (1982), 143–152. https://doi.org/10.1016/0167-6423(82)90012-0

[48] Dimitris Mouris, Christopher Patton, Hannah Davis, Pratik Sarkar, and Nektarios Georgios Tsoutsos. 2025. Mastic: Private Weighted Heavy-Hitters and Attribute-Based Metrics. *Proceedings on Privacy Enhancing Technologies* 2025, 1 (2025), 290–319. https://doi.org/10.56553/popets-2025-0017

[49] Netflix. 2009. Netflix Prize Dataset. https://archive.org/download/nf_prize_dataset.tar.

[50] Landon Curt Noll, Kiem-Phong Vo, Donald E. Eastlake 3rd, and Tony Hansen. 2026. The FNV Non-Cryptographic Hash Algorithm. RFC 9923, Independent Submission, Informational. https://doi.org/10.17487/RFC9923

[51] Stavros Papadopoulos, Yin Yang, and Dimitris Papadias. 2007. CADS: Continuous Authentication on Data Streams. In *Proceedings of the 33rd International Conference on Very Large Data Bases (VLDB)*. ACM, 135–146.

[52] Charalampos Papamanthou, Elaine Shi, Roberto Tamassia, and Ke Yi. 2013. Streaming Authenticated Data Structures. In *Advances in Cryptology – EUROCRYPT 2013*. Springer, 353–370.

[53] Heejin Park, Shuang Zhai, Long Lu, and Felix Xiaozhu Lin. 2019. StreamBox-TZ: Secure Stream Analytics at the Edge with TrustZone. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. 537–554.

[54] Greg Pass, Abdur Chowdhury, and Cayley Torgeson. 2006. AOL User Click Collection (AOL-user-ct-collection). Real-world website-access log with user IDs and click times.

[55] Polygon Labs. 2024. Plonky3: A Toolkit for Implementing Polynomial IOPs over Small Fields. https://github.com/Plonky3/Plonky3 Accessed 2026-05-03.

[56] Danica Porobic, Ippokratis Pandis, Miguel Branco, Pınar Tözün, and Anastasia Ailamaki. 2012. OLTP on Hardware Islands. *Proceedings of the VLDB Endowment* 5, 11 (2012), 1447–1458. https://doi.org/10.14778/2350229.2350260

[57] Christian Priebe, Kapil Vaswani, and Manuel Costa. 2018. EnclaveDB: A Secure Database Using SGX. In *2018 IEEE Symposium on Security and Privacy (SP)*. 264–278. https://doi.org/10.1109/SP.2018.00025

[58] Zipeng Qiu, Wenjie Qu, Jiaheng Zhang, and Binhang Yuan. 2026. V3DB: Audit-on-Demand Zero-Knowledge Proofs for Verifiable Vector Search over Committed Snapshots. *Proceedings of the VLDB Endowment* 19, 10 (2026), 2604–2616. https://doi.org/10.14778/3828612.3828618

[59] Mayank Rathee, Yuwen Zhang, Henry Corrigan-Gibbs, and Raluca Ada Popa. 2024. Private Analytics via Streaming, Sketching, and Silently Verifiable Proofs. In *2024 IEEE Symposium on Security and Privacy (SP)*. 3072–3090. https://doi.org/10.1109/SP54263.2024.00245 Full version: IACR Cryptology ePrint Archive, Report 2024/666.

[60] Srinath Setty, Sebastian Angel, Trinabh Gupta, and Jonathan Lee. 2018. Proving the Correct Execution of Concurrent Services in Zero-Knowledge. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 339–356.

[61] Guy L. Steele Jr., Doug Lea, and Christine H. Flood. 2014. Fast Splittable Pseudorandom Number Generators. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA)*. ACM, 453–472. https://doi.org/10.1145/2660193.2660195

[62] Janwillem Swalens, Lode Hoste, Emad Heydari Beni, and Lieven Trappeniers. 2024. zkStream: A Framework for Trustworthy Stream Processing. In *Proceedings of the 25th International Middleware Conference*. 252–265. https://doi.org/10.1145/3652892.3700763

[63] Paul Valiant. 2008. Incrementally Verifiable Computation or Proofs of Knowledge Imply Time/Space Efficiency. In *Theory of Cryptography Conference (TCC)*. Springer, 1–18.

[64] Jo Van Bulck, Marina Minkin, Ofir Weisse, Daniel Genkin, Baris Kasikci, Frank Piessens, Mark Silberstein, Thomas F. Wenisch, Yuval Yarom, and Raoul Strackx. 2018. Foreshadow: Extracting the Keys to the Intel SGX Kingdom with Transient Out-of-Order Execution. In *27th USENIX Security Symposium (USENIX Security 18)*. 991–1008.

[65] Stephan van Schaik, Alex Seto, Thomas Yurek, Adam Batori, Bader AlBassam, Daniel Genkin, Andrew Miller, Eyal Ronen, Yuval Yarom, and Christina Garman. 2024. SoK: SGX.Fail: How Stuff Gets eXposed. In *2024 IEEE Symposium on Security and Privacy (SP)*. 4143–4162. https://doi.org/10.1109/SP54263.2024.00260

[66] Zhaoyu Wang, Pingchuan Ma, Zhantong Xue, Yanbo Dai, Zhenlan Ji, and Shuai Wang. 2025. Privacy-Preserving and Verifiable Causal Prescriptive Analytics. *Proceedings of the ACM on Management of Data* 3, 6, Article 350 (2025). https://doi.org/10.1145/3769815

[67] Ke Yi, Feifei Li, Graham Cormode, Marios Hadjieleftheriou, George Kollios, and Divesh Srivastava. 2009. Small Synopses for Group-By Query Verification on Outsourced Data Streams. *ACM Transactions on Database Systems* 34, 3 (2009), 15:1–15:42. https://doi.org/10.1145/1567274.1567277

[68] Jianzhe Yu, Feng Han, Qi Dong, Qiyao Luo, Weiran Liu, Lin Qu, and Ke Yi. 2026. Sketch-Based Secure Query Processing for Streaming Data. *Proceedings of the ACM on Management of Data* 4, 3, Article 233 (2026). https://doi.org/10.1145/3802110

[69] Bing Zhang, Vadym Doroshenko, Peter Kairouz, Thomas Steinke, Abhradeep Thakurta, Ziyin Ma, Eidan Cohen, Himani Apte, and Jodi Spacek. 2024. Differentially Private Stream Processing at Scale. *Proceedings of the VLDB Endowment* 17, 12 (2024), 4145–4158. https://doi.org/10.14778/3685800.3685833

[70] Yupeng Zhang, Daniel Genkin, Jonathan Katz, Dimitrios Papadopoulos, and Charalampos Papamanthou. 2017. A Zero-Knowledge Version of vSQL. Cryptology ePrint Archive, Paper 2017/1146. https://eprint.iacr.org/2017/1146

[71] Fuheng Zhao, Dan Qiao, Rachel Redberg, Divyakant Agrawal, Amr El Abbadi, and Yu-Xiang Wang. 2022. Differentially Private Linear Sketches: Efficient Implementations and Applications. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*. 12691–12704. https://doi.org/10.52202/068431-0922

[72] Wenting Zheng, Ankur Dave, Jethro G. Beekman, Raluca Ada Popa, Joseph E. Gonzalez, and Ion Stoica. 2017. Opaque: An Oblivious and Encrypted Distributed Analytics Platform. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. 283–298.

[73] Wenchao Zhou, Yifan Cai, Yanqing Peng, Sheng Wang, Ke Ma, and Feifei Li. 2021. VeriDB: An SGX-based Verifiable Database. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*. 2182–2194. https://doi.org/10.1145/3448016.3457308

[74] Zeying Zhu, Clarence Lam, Alexander Frolov, Ian Miers, and Zaoxing Liu. 2026. Zero-Knowledge Cloud Analytics. In *Proceedings of the ACM SIGCOMM 2026 Conference*. 1299–1314. https://doi.org/10.1145/3789240.3829157

# A CRYPTOGRAPHIC AND CIRCUIT DETAILS

## A.1 Proof Backend and Cost

zkSketch uses a FRI-based zkSTARK [8] implemented in Plonky3 (Section 3.1), with Poseidon2 hashing [31]. Its *challenge field* is the degree-4 extension of the BabyBear prime field $\mathbb{F}$ of our Plonky3 backend, denoted by $\mathbb{K} \supseteq \mathbb{F}$, with $|\mathbb{K}| \approx 2^{124}$.

Fiat–Shamir [26] makes the proof non-interactive by computing the verifier's *challenges* from transcript hashes, with the hash modeled as a random oracle. Challenges lie in $\mathbb{K}$; a false polynomial identity can hold at only a small fraction of the possible challenge values. Appendix A.2 bounds the resulting soundness error for the table checks, separately from the backend protocol's error.

Proof size and verification time are polylogarithmic in the number of circuit rows (Section 3.1). The circuit uses arithmetic modulo $|\mathbb{F}|$, so integer comparisons and sums require suitable bounds. To check $x \le y$, it range-checks $y - x \in [0, 2^B)$ for a declared width $B$. The bounds on $x$ and $y$ must prevent a negative difference from wrapping into this interval. Likewise, a sum interpreted as an integer must stay below $|\mathbb{F}|$. Where the circuit does not enforce these bounds, they are stated as input assumptions (Appendices B.1.1 and B.4).

**Composition.** A composition step (Definition 6.2) verifies two child proofs inside its circuit. The child proofs and their shared commitment are private inputs; the children's verification keys are fixed in the circuit. The root's verification key, derived from the public profile, therefore binds every circuit in the tree. Since a circuit cannot fix its own key, the composition step is one template, instantiated per sketch family and pair of child shapes; all keys can be precomputed from the profile. The security argument assumes that the backend's knowledge soundness carries through recursive verification: an accepted proof implies that the prover knows a valid witness.

The batch hash forms a hash tree [45], with its shape determined by the composition schedule in the profile. A batch proof exposes $\mathsf{H}(\mathsf{LEAF}, t, \mathrm{Com}(B_t))$ for the profile's cryptographic hash $\mathsf{H}$. A composition joining $[i, h]$ and $[h+1, j]$ hashes $(\mathsf{MERGE}, i, h, j)$ together with the two child hashes. These tags and split indices let the verifier recompute the hash unambiguously from the log. Indices have an unambiguous field representation because $t < |\mathbb{F}|$.

## A.2 Circuit Building Blocks

The same-rows check of Section 7.1 is LogUp [35], a lookup argument based on logarithmic derivatives. For tables $U = (u_1, \ldots, u_n)$ and $U' = (u'_1, \ldots, u'_n)$, it checks the randomized identity

$$\sum_{i=1}^{n} \frac{1}{\gamma - \mathrm{enc}(u_i)} = \sum_{i=1}^{n} \frac{1}{\gamma - \mathrm{enc}(u'_i)},$$

Here, enc encodes the complete row, including a label that distinguishes separate table relations. The challenge $\gamma$ is drawn after the tables are committed.

Let $n$ be the public table height and $B$ the declared value width. Opening a committed table requires re-hashing its $n$ rows and costs $\Theta(n)$ circuit rows. Hashing a key takes one Poseidon2 permutation, or $\Theta(1)$ rows. An index, rank, or bit taken from a hash output comes from decomposing a field element, and the circuit checks that the decomposition encodes an integer below $|\mathbb{F}|$, so each derived value is unique. The four building blocks have the following costs.

- *Same-rows* (LogUp) adds one fractional term per entry of either table, for a cost of $\Theta(n)$. All same-rows checks in a circuit share one LogUp sum. A row packs up to four fields into one element of $\mathbb{K}$. A wider row, such as a Greenwald–Khanna tuple, spans two elements that both carry the row's position, so parts of different rows cannot be swapped; each element counts as one entry below. With at most $2^{25}$ entries per sum, a false claim passes with probability below $2^{-96}$. LogUp counts multiplicities modulo $|\mathbb{F}|$; the limit of $2^{24}$ rows per table keeps these counts below the modulus.
- *Range check* looks up each byte in the public table $\{0, \ldots, 255\}$ and checks that the bytes reconstruct the value. When $8 \nmid B$, an additional lookup bounds the shifted top byte. The cost is $\Theta(\lceil B/8 \rceil)$ per value. Circuits that use $B$ Boolean bits instead cost $\Theta(B)$ per value.
- *Sorted copy* combines a same-rows check with a range check on each of the $n - 1$ adjacent differences, for a cost of $\Theta(n\lceil B/8 \rceil)$.
- *Group-by* fixes each run-end flag by checking adjacent-key equality, using one supplied inverse per row. Each row also contributes one step to the running aggregate. A sum costs $\Theta(n)$ rows; a running maximum adds two range checks per row.

# B UPDATE AND QUERY CIRCUITS IN DETAIL

Each sketch has an *update circuit* for a batch and a *query circuit* for an answer. The update circuit checks $S_t = \mathrm{UpdateBatch}(S_{t-1}, B_t)$, with the old and new states bound by $\mathrm{Com}(S_{t-1})$ and $\mathrm{Com}(S_t)$. Its batch proof is $\pi_t$ (Definition 6.1). The query circuit opens a certified state $S_t$ against $\mathrm{Com}(S_t)$ and checks the reported answer against the sketch's query rule (Section 7.4).

Each subsection presents the update checks followed by the query checks. Since Sections 7.2 and 7.3 already give the full Count-Min and Misra–Gries update circuits, their subsections here cover only the remaining details. Query circuits, like update circuits, are oblivious: their size and structure depend only on the profile and the query's public parameters.

## B.1 Count-Min

*B.1.1 Update Circuit.* Section 7.2 gives the complete update circuit. The discussion below states its input bound and explains the tradeoffs with alternative designs.

**Input bound.** The circuit adds counts in $\mathbb{F}$. These additions agree with integer updates only if every counter, including all contributions it receives, stays below $|\mathbb{F}|$. This also bounds every partial run sum. The query's range-checked comparisons (Appendix B.1.2) need every counter below $2^B$ for the declared comparison width $B$, where $2^{B+1} \le |\mathbb{F}|$. The circuit does not range-check the counters, so the deployment must guarantee these conditions through the profile's bound on the total weight of the records. Otherwise, a proof certifies only the update modulo $|\mathbb{F}|$.

**Alternative designs.** A Merkle tree over each sketch row authenticates a counter along a path and computes the new root after the update. The path also binds the untouched subtrees, so the circuit need not check a balance over the whole array. This costs $\Theta(d \log w)$ hashes per record, plus routing each access to its path.

It avoids the $\Theta(dw)$ term for small batches but re-authenticates shared paths in large batches. The cheaper design therefore depends on $m$, $w$, and the relative costs of hashing and memory checking.

A zkVM, as in zk-Analytics [74], proves execution of the per-record update program and pays for its instructions and memory accesses. Section 8.2 compares this approach with zkSketch end to end.

A third option is to skip grouping and install the *md* accesses one at a time with timestamps, as the Bloom filter circuit does (Appendix B.3). This also costs $\Theta(md + dw)$ rows. Count-Min needs neither grouping nor timestamps: addition is invertible, so the untagged balance over the *md* raw accesses already forces $S_t[a] = S_{t-1}[a] + \sum_{a_{p,i}=a} v_p$, since equal multisets at an address have equal sums. Grouping matters for a maximum or an OR, where it leaves at most one patch entry per cell, so HyperLogLog needs only a one-bit tag (Appendix B.2).

*B.1.2 Query Circuit.* A point query for key $k$ returns the smallest of its $d$ counters, $\min_i S_t[i, h_i(k)]$. The circuit opens $S_t$ and hashes $k$, fixing which counters must be read. For each sketch row $i$, the prover supplies a Boolean *selector* column over its $w$ counters. The circuit checks that the column contains exactly one 1, at position $h_i(k)$. Weighting the counters by this column and summing them yields $S_t[i, h_i(k)]$.

The answer must be no larger than any selected counter and must equal at least one of them. As in Section 7.4, range checks on the differences enforce the first condition, and the vanishing product enforces the second. Opening the array and selecting the counters each take $\Theta(dw)$ rows.

## B.2 HyperLogLog

*B.2.1 Update Circuit.* Each key $k$ contributes a rank $\mu(k)$ to register $h(k)$, as defined in Section 7.2.3. The rank is the position of the first 1 in the key's hash string. The circuit checks that each register retains the largest rank it has received.

**HyperLogLog update circuit.**
**Input.** Public label $\text{pub}_t$ and capacities $m, s$; private batch $B_t$, old and new register arrays $S_{t-1}, S_t$, their commitment openings, and intermediate tables.
**Output.** A proof $\pi_t$ certifying the new state commitment.
**Verification target.** The batch and arrays open to their commitments, and every register satisfies

$$S_t[a] = \max\big(S_{t-1}[a],\ \mu_a\big), \qquad \mu_a = \max_{p:\, h(k_p)=a} \mu(k_p),$$

with $\mu_a = 0$ if no record reaches register $a$.

**Key idea.** The circuit follows Count-Min's five phases with $d = 1$ and $w = s$. It replaces amounts with ranks and addition with max, then installs the resulting patch using the before/after tag of Section 7.2.3.

**Phase 1: Binding the batch and registers.** The circuit opens $B_t$, $S_{t-1}$, and $S_t$ against $\text{pub}_t$.

**Phase 2: Deriving the contributions.** For each record $p$, the circuit computes $h(k_p)$ and $\mu(k_p)$ and forms the contribution $U_p = (h(k_p), \mu(k_p))$. Computing both values in the circuit prevents the prover from choosing a different register or rank. A partial batch is padded by repeating one of its real keys, which leaves all maxima unchanged.

**Phase 3: Grouping equal registers.** A sorted-copy check certifies that $U^{\uparrow}$ contains the rows of $U$ in register order. Its rows $(a_z, \mu_z^{\uparrow})$ therefore place each register's contributions in one run.

**Phase 4: Computing the patch.** A group-by check with a running maximum,

$$c_1 = \mu_1^{\uparrow}, \qquad c_z = \max\big(\mu_z^{\uparrow},\ (1 - f_{z-1})\, c_{z-1}\big) \quad (z > 1),$$

places each run's largest rank on its flagged row. The circuit checks each maximum by requiring it to be at least both operands and equal to one of them. The flagged rows form the patch $P = \{(a, \mu_a)\}$, with one entry per touched register; the remaining rows become dummies.

**Phase 5: Installing with a before/after tag.** For each patch entry $(a, \mu_a)$, the prover supplies the read $r_a$. One same-rows check certifies the tagged balance of Section 7.2.3, which holds separately for each register, as in Count-Min. At a touched register, the write $(a, \max(r_a, \mu_a))$ is the only tag-1 element on the left. The new register must therefore carry tag 1 and match the write. This leaves the old register as the only tag-0 element that can match the read, forcing $r_a = S_{t-1}[a]$ and $S_t[a] = \max(S_{t-1}[a], \mu_a)$. An untouched register has no read or write, so its value is unchanged.

**Cost.** Each phase is a linear pass, so the circuit uses

$$\underbrace{\Theta(m+s)}_{\text{1: bind}} + \underbrace{\Theta(m)}_{\text{2: derive}} + \underbrace{\Theta(m)}_{\text{3: group}} + \underbrace{\Theta(m)}_{\text{4: patch}} + \underbrace{\Theta(m+s)}_{\text{5: install}} = \Theta(m+s)$$

rows per batch at fixed widths, or $\Theta(1 + s/m)$ per record.

**Correctness and obliviousness.** The commitment openings fix the inputs, and the in-circuit hashes fix every contribution. The sorted-copy, flag, and maximum checks ensure that each patch entry contains its register's largest rank. The tagged balance then enforces $S_t[a] = \max(S_{t-1}[a], \mu_a)$ at every touched register and $S_t[a] = S_{t-1}[a]$ elsewhere. Unlike Count-Min, this circuit forms no sum: it only compares range-checked ranks and register values, so it needs no input bound. Every table has a public height ($m$ or $s$). The set of registers that change therefore affects only values and flags, not the circuit's structure.

*B.2.2 Query Circuit.* The query estimates the number of distinct keys as

$$\frac{\alpha_s\, s^2}{Z}, \qquad Z = \sum_a 2^{-S_t[a]},$$

for the public bias-correction constant $\alpha_s$ [28]; no query key is needed. The circuit opens $S_t$ and computes $Z$ in fixed point. For each register, a selector uses the register value to pick $2^{-S_t[a]}$ from a public table, and the circuit sums these terms. The prover supplies the quotient *res* and remainder *rem*, which the circuit checks through

$$res \cdot Z + rem = \alpha_s s^2, \qquad 0 \le rem < Z,$$

over the integers, with *res* range-checked so that the identity cannot wrap modulo $|\mathbb{F}|$. The profile fixes integer approximations of $\alpha_s s^2$ and of the table entries $2^{-x}$ at public power-of-two scales; the declared query is the quotient of these rounded values, which *res* gives in fixed point. This raw estimate is reliable once it exceeds

$5s/2$; the declared query omits HyperLogLog's small-range correction, linear counting over the zero registers [28]. The circuit uses $\Theta(s)$ rows.

## B.3 Bloom Filter

*B.3.1 Update Circuit.* The partitioned filter of Section 7.2.3 has $k$ slices of $W$ bits, stored as $kW/8$ bytes. The update circuit checks that each record sets its addressed bits while preserving all other bits.

**Bloom filter update circuit.**
**Input.** Public label $\mathsf{pub}_t$ and dimensions $m, k, W$; private batch $B_t$, old and new byte arrays $S_{t-1}, S_t$, their commitment openings, and intermediate tables.
**Output.** A proof $\pi_t$ certifying the new state commitment.
**Verification target.** The batch and arrays open to their commitments, and for every slice $j$ and bit position $b$,

$$S_t[j,b] = S_{t-1}[j,b] \vee \mathbf{1}\big[\exists \,\mathrm{real}\; p :\; h_j(k_p) = b\big].$$

**Key idea.** Several accesses may touch the same byte, and each must preserve the other seven bits when setting its own. Grouping accesses by byte would require a bitwise OR, which the group-by check does not compute. Grouping by bit would still leave several writes to one byte. The circuit instead applies the $mk$ (record, slice) accesses in a fixed public order $z = 1, \ldots, mk$. Each access reads its byte, splits it at the addressed bit, and writes it back with that bit set. Timestamps order the reads and writes to each byte (Section 7.2.3).
**Phase 1: Binding the batch and filters.** The circuit opens $B_t$, $S_{t-1}$, and $S_t$ against $\mathsf{pub}_t$ and checks that every old and new value is a byte.
**Phase 2: Routing the accesses.** Access $z$ sends record $p$ to slice $j$. The circuit computes $h_j(k_p)$ and splits it as $8q + i$, where $q < W/8$ indexes a byte within the slice and $i < 8$ indexes a bit within that byte. The access therefore touches byte $u_z = jW/8 + q$ at the bit of weight $e_z = 2^i$.
**Phase 3: Setting the bit.** The prover supplies the byte $y_z$ read by access $z$ and its decomposition at the addressed bit:

$$y_z = (2H_z + c_z)\, e_z + L_z, \qquad L_z < e_z, \quad c_z \in \{0,1\},$$

with $H_z$ and $L_z$ checked to be bytes. Phase 4 checks that $y_z$ is the byte's current value. Because all terms are small integers, the split is unique and $c_z$ is the addressed bit. The circuit forms the written byte $y'_z = y_z + \mathit{real}_z(1 - c_z)\, e_z$, where $\mathit{real}_z$ marks a real record. A real access sets its bit without carrying into the next one; a dummy access leaves the byte unchanged.
**Phase 4: Installing with timestamps.** Access $z$ writes at time $z$ and reads at a supplied earlier time $z - 1 - \delta_z$, with $\delta_z \geq 0$ range-checked. Each byte $u$ starts with value $S_{t-1}[u]$ at time 0 and ends with value $S_t[u]$ at a supplied final time $\psi_u$. One same-rows check certifies

$$\{(u, S_{t-1}[u], 0)\}_u \uplus \{(u_z, y'_z, z)\}_z = \{(u, S_t[u], \psi_u)\}_u \uplus \{(u_z, y_z, z{-}1{-}\delta_z)\}_z.$$

The write times are distinct, and every read precedes its own write. Together with the balance, these conditions make each read return the byte's current value in access order and make the new state retain its last write [13]. In particular, a read cannot match its own write: such a match would let an access cancel itself out and leave its bit unset.
**Cost.** The circuit uses

$$\underbrace{\Theta(m + kW/8)}_{\text{1: bind}} + \underbrace{\Theta(mk)}_{\text{2: route}} + \underbrace{\Theta(mk)}_{\text{3: set}} + \underbrace{\Theta(mk + kW/8)}_{\text{4: install}} = \Theta(mk + kW/8)$$

rows per batch at fixed widths, or $\Theta(k + kW/(8m))$ per record. The timestamp range checks limit a batch to $mk < 2^{16}$ accesses. Larger batches, including the default $m = 2^{14}$ with $k = 7$, use an earlier circuit that stores one bit per cell. This circuit routes each key as Count-Min does, combines contributions by OR in place of addition, and installs the patch with HyperLogLog's before/after tag. It costs $\Theta(mk + kW)$ rows per batch and remains oblivious because its tables also have public heights.
**Correctness and obliviousness.** The commitment openings fix the inputs, and the in-circuit hashes fix the byte and bit of each access. The timestamped balance makes each read return the latest value of its byte, while the decomposition ensures that each real access sets exactly its addressed bit. The new filter thus contains all bits set in the old filter or by the batch; untouched bytes retain their values. Every value stays a byte, so no input bound is needed. The table heights ($m$, $mk$, or $kW/8$) are public, and the bits set by the batch affect only values, not the circuit's structure.

*B.3.2 Query Circuit.* For a queried key $\hat{k}$, the filter answers $\bigwedge_j S_t[j, h_j(\hat{k})]$. An answer of 0 means that $\hat{k}$ was never inserted; an answer of 1 indicates possible membership, subject to the filter's false-positive rate. The circuit opens $S_t$, hashes $\hat{k}$, and routes it as in Phase 2. For each slice $j$, a selector over its $W/8$ bytes picks the addressed byte, and the decomposition from Phase 3 extracts the addressed bit. The circuit releases the product of the $k$ bits. Opening the filter and selecting the bytes each take $\Theta(kW/8)$ rows. In the filter with one bit per cell, the selector reads each bit directly, at a cost of $\Theta(kW)$ rows.

## B.4 Misra–Gries

*B.4.1 Update Circuit.* Section 7.3 gives the complete circuit. Here we specify the stored order and input bound, and recall the relation to SpaceSaving.
**Stored order.** The circuit stores $G^{\downarrow}$ in reverse order, lightest first, and takes its last $s$ rows as the state. Thus, $S_t$ contains dummy slots followed by survivors from lightest to heaviest. The query circuit checks and uses this order.
**Input bound.** The circuit range-checks every count it opens, keeps, or reduces. It does not range-check the running sums used to merge a key's rows, so a sum could wrap modulo $|\mathbb{F}|$ and fall back within the allowed range. A merged count is exact, and the count-order check of Phase 3 is sound, as long as the key's old count plus its batch amounts stays below $2^B$, where $2^{B+1} \leq |\mathbb{F}|$. As for Count-Min, the deployment must guarantee this bound because the circuit does not check it; the profile's bound on the total weight of the records does so and also bounds the query's released sum.
**SpaceSaving.** SpaceSaving [46] with $s + 1$ counters carries the same information as Misra–Gries with $s$: on the same stream processed one record at a time, subtracting its minimum counter from each of its counters and omitting the zeros yields the Misra–Gries

summary [1]. The batch rule certified here can leave a different summary with the same error bound, so SS is only the implementation's name.

*B.4.2 Query Circuit.* The query returns the total count of the $K$ heaviest stored keys, $\sum_{j \le K} c_{(j)}$, for a public $K$ (Section 7.4). Because the state is stored from lightest to heaviest, these keys occupy the last $\min(K, s^*)$ slots of $S_t$, where $s^*$ is the number of real slots. The circuit opens $S_t$ and checks that the real slots form a suffix, with nondecreasing counts and distinct keys in adjacent slots. A 0/1 column marks the selected slots: it contains $\min(K, s^*)$ ones following all its zeros, so it selects exactly the last $\min(K, s^*)$ slots, all real. The circuit releases the sum of their counts. The update circuit guarantees distinct keys across the whole state, since its grouping stores each key at most once in every certified state. The query circuit uses $\Theta(s)$ rows.

## B.5 KMV

*B.5.1 Update Circuit.* For the KMV summary of Section 7.3.3, let $\text{bottom}_s(X)$ denote the $s$ smallest distinct values of $X$, or all distinct values if fewer exist. The values are stored in ascending order and padded with dummies to $s$ slots. The update circuit certifies the following relation.

**KMV update circuit.**
**Input.** Public label $\text{pub}_t$ and capacities $m, s$; private batch $B_t$, old and new summaries $S_{t-1}, S_t$, their commitment openings, and intermediate tables.
**Output.** A proof $\pi_t$ certifying the new state commitment.
**Verification target.** The batch and summaries open to their commitments, and

$$S_t = \text{bottom}_s\big(S_{t-1} \cup h(B_t)\big),$$

where $h(B_t)$ contains the hashes of the real records only.

Since $\text{bottom}_s(\text{bottom}_s(X) \cup Y) = \text{bottom}_s(X \cup Y)$, this batch rule gives the same state as inserting the records one at a time.
**Key idea.** The circuit adapts the Misra–Gries construction with the two changes described in Section 7.3.3. It keeps a value when its rank among the distinct values is at most $s$. These values need not occupy the first $s$ rows, so stable compaction moves them into the output slots.
**Phase 1: Binding and hashing.** The circuit opens $B_t$, $S_{t-1}$, and $S_t$ against $\text{pub}_t$ and computes each key's hash. The old values and the batch's hashes form a candidate table of $n = s + m$ rows, each with a real/dummy flag.
**Phase 2: Sorting the candidates.** A sorted-copy check places the real values first, in ascending order $u_1 \le u_2 \le \cdots$. The check includes each row's flag and ensures that no occurrence is added, dropped, or changed.
**Phase 3: Selecting the smallest distinct values.** Adjacent comparisons determine the first-occurrence flags $d_i$. Their running sum $\sigma_i$ gives each value its rank among the distinct values, as in Section 7.3.3. A value is kept at its first occurrence if $\sigma_i \le s$. To avoid comparing every rank with $s$, the prover supplies an eligibility column $\omega_i$. The circuit checks that this column is Boolean and nonincreasing, $(1 - \omega_i)\,\omega_{i+1} = 0$, and sets the keep flags to $b_i = d_i\,\omega_i$. With $\nu = \sum_i d_i$ distinct real values, two range checks and a product constrain the number of kept values to $\min(s, \nu)$:

$$\kappa = \sum_i b_i, \qquad \kappa \le s, \quad \kappa \le \nu, \quad (s - \kappa)(\nu - \kappa) = 0.$$

Because $\omega$ consists of ones followed by zeros and the values are sorted, these checks select exactly the $\kappa$ smallest distinct values.
**Phase 4: Placing the survivors.** The rank $\sigma_i$ of each kept value gives its output slot. The circuit represents each kept row by $(u_i, \sigma_i)$ and every other row by a fixed dummy $\perp$. For the committed output, it forms $(S_t[j], j)$ at each of the $\kappa'$ real slots and $\perp$ at the others. With both sides padded to height $n$, the same-rows check

$$\{(u_i, \sigma_i) : b_i = 1\} \uplus \{\perp\}^{n-\kappa} = \{(S_t[j], j) : \text{slot } j \text{ real}\} \uplus \{\perp\}^{n-\kappa'},$$

forces the real slots to be exactly $1, \ldots, \kappa$ and puts each kept value in its slot.
**Cost.** The circuit uses

$$\underbrace{\Theta(n)}_{\text{1: bind}} + \underbrace{\Theta(n)}_{\text{2: sort}} + \underbrace{\Theta(n)}_{\text{3: select}} + \underbrace{\Theta(n)}_{\text{4: place}} = \Theta(s + m)$$

rows per batch at fixed widths, or $\Theta(1 + s/m)$ per record.
**Correctness and obliviousness.** The commitment openings and hash computation determine the candidates. The sorted copy contains exactly their real occurrences in ascending order. The checked flags, ranks, and kept count then select the $\min(s, \nu)$ smallest distinct values. Compaction stores them in the first $\kappa$ slots in ascending order and fills the remaining slots with dummies, as required by the verification target. Counts and ranks never exceed $n$, so no input bound is needed. Every table has a public height ($m$, $s$, or $n$). Thus, the number of distinct values and survivors affects the values and flags but not the circuit's structure.

*B.5.2 Query Circuit.* When the summary contains $s$ values, KMV estimates the number of distinct keys as $(s - 1)H/v_s$, where $v_s$ is the largest kept value and $H$ is the size of the hash range. With fewer than $s$ values, it returns their count, which is exact unless two distinct keys share a hash value [2]. The circuit opens $S_t$ and checks that its real values form a strictly increasing prefix. This check excludes duplicates, which would lower $v_s$ and inflate the estimate. If the summary is full, the circuit certifies $\lfloor (s - 1)H/v_s \rfloor$ using a supplied quotient and remainder, as for HyperLogLog. Otherwise, it releases the number of real slots. The circuit uses $\Theta(s)$ rows.

## B.6 Greenwald–Khanna

*B.6.1 Update Circuit.* The two-sided tuples of Section 7.3.3 also record each value's arrival position $\tau$, which determines the direction of fusion [33]. In full, a tuple $(e, g_1, g_2, \Delta_1, \Delta_2, \tau)$ covers $e$ itself, $g_1$ records below it, and $g_2$ above it, subject to the invariant stated in that section. The summary holds at most $s'$ tuples. Every update table has the public height $n = s' + m$; the profile requires this height to be even so that the rows can be paired in Phase 3. The update circuit certifies the following relation.

**GK update circuit.**
**Input.** Public label $\text{pub}_t$, capacities $m, s'$, and accuracy $\varepsilon$; private prior count $N_{t-1}$, threshold $\theta_t$, batch $B_t$, old and new summaries $S_{t-1}, S_t$, their commitment openings, and intermediate tables.

**Output.** A proof $\pi_t$ certifying the new state commitment.
**Verification target.** The batch and summaries open to their commitments; $N_{t-1}$ is the number of records that $S_{t-1}$ covers, and $\theta_t = \lfloor \varepsilon N_t \rfloor$ for $N_t = N_{t-1} + n_t$, where $n_t$ counts the batch's real records; and $S_t$ is the declared batch rule applied to $S_{t-1}$ and $B_t$ with threshold $\theta_t$.

The batch rule inserts the records as fresh tuples, initializes them from their neighbors, and fuses fixed adjacent pairs for $L_{\mathrm{f}} = \lceil \log_2 n \rceil$ rounds.

**Key idea.** The circuit adapts the Misra–Gries construction by replacing the keep decision with a fusion decision (Section 7.3.3). Each of the $L_{\mathrm{f}}$ fusion rounds is followed by KMV's stable compaction. A check on total coverage binds the fusion threshold to the committed history.

**Phase 1: Binding and sorting the union.** The circuit opens $B_t$, $S_{t-1}$, and $S_t$ against $\mathrm{pub}_t$. It converts the batch's real records into fresh tuples $(e, 0, 0, 0, 0, \tau)$ with arrival positions $\tau = N_{t-1}+1, \ldots, N_t$ in stream order. It also checks that the old positions are at most $N_{t-1}$, making every fresh tuple newer than every old tuple. A sorted-copy check orders the union by $(e, \tau)$, with dummies last. Each row carries a liveness bit $\ell_i$: 1 for a tuple and 0 for a dummy. The live rows therefore form a prefix. A tuple absorbed in Phase 3 becomes dead.

**Phase 2: Initializing the fresh tuples.** Each fresh tuple receives $\Delta_1$ from its nearest old successor's $g_1 + \Delta_1$ and $\Delta_2$ from its nearest old predecessor's $g_2 + \Delta_2$. A field is set to zero if the corresponding neighbor is absent. The circuit fills these fields by scanning in both directions and copying the relevant bound until it reaches another old row. This produces the initialized table $G^{(0)}$.

**Phase 3: Fusing fixed pairs.** In round $\lambda$, the circuit tests the fixed adjacent pairs $(2i-1, 2i)$ of $G^{(\lambda-1)}$. For each pair $(e, e')$, a checked comparison of arrival positions identifies the newer tuple. This tuple can fuse into the older one if the older tuple's affected side remains within $\theta_t$. If $e$ is newer, it fuses into $e'$ (first line); otherwise $e'$ fuses into $e$ (second line):

$$g_1'(e') = g_1(e) + g_2(e) + g_1(e') + 1, \quad \phi = \mathbf{1}\big[g_1'(e') + \Delta_1(e') \le \theta_t\big];$$
$$g_2'(e) = g_1(e') + g_2(e') + g_2(e) + 1, \quad \phi = \mathbf{1}\big[g_2'(e) + \Delta_2(e) \le \theta_t\big].$$

A pair fuses when $\phi = 1$ and its right row is live. The live-prefix property ensures that the left row is then live as well. The circuit clears the absorbed row's liveness bit.

**Phase 4: Compacting the survivors.** Fusion changes coverage but preserves the survivors' values, arrival positions, and order. A survivor's next slot therefore depends only on which rows remain live. Prefix counts assign a destination to every row, placing live rows in order at the front and dead rows at the back:

$$dst_i = \begin{cases} 1 + \sum_{j<i} \ell_j, & \ell_i = 1, \\ n - \sum_{j<i}(1 - \ell_j), & \ell_i = 0. \end{cases}$$

The circuit uses KMV's stable compaction to move each updated tuple and its liveness bit to the assigned slot in $G^{(\lambda)}$. Every element carries its round number and destination, preventing it from moving to another round's table or another slot. Here, compaction retains the dead rows, keeping every table at height $n$. The live rows remain sorted by $(e, \tau)$, so no further sorting is needed.

**Phase 5: Binding the threshold and the output.** The circuit must prevent the prover from overstating $N_{t-1}$ to raise $\theta_t$ and allow more fusions. Fusion preserves total coverage, so the total coverage $\mathrm{mass}(\cdot)$ of the live tuples increases by exactly the batch's $n_t$ real records, independently of the supplied threshold. The circuit checks

$$N_t = N_{t-1} + n_t = \mathrm{mass}(S_t),$$
$$N_t = \varepsilon^{-1}\theta_t + r_t, \qquad 0 \le r_t < \varepsilon^{-1},$$

and range-checks $\theta_t$. The profile requires $\varepsilon^{-1}$ to be an integer, so these checks bind $N_{t-1}$ to the coverage of $S_{t-1}$ and $\theta_t$ to $\lfloor \varepsilon N_t \rfloor$. To form the output, the circuit checks that slot $s' + 1$ of the last table is dead and replaces the dead rows among the first $s'$ with a fixed dummy tuple. A fixed split then binds the result to $S_t$. If a batch leaves more than $s'$ survivors, the prover cannot produce an accepted proof; the circuit never discards an excess survivor.

**Cost.** Each phase takes a linear pass over its table. Phases 3 and 4 repeat in every fusion round, giving

$$\underbrace{\Theta(n)}_{\text{1: bind}} + \underbrace{\Theta(n)}_{\text{2: init}} + \underbrace{\Theta(nL_{\mathrm{f}})}_{\text{3: fuse}} + \underbrace{\Theta(nL_{\mathrm{f}})}_{\text{4: compact}} + \underbrace{\Theta(n)}_{\text{5: output}} = \Theta\big((s' + m)L_{\mathrm{f}}\big)$$

rows per batch at fixed widths, or $\Theta(\log s')$ per record at $m = \Theta(s')$.

**Correctness and obliviousness.** Appendix B.6.2 proves that these checks enforce the declared batch rule with $\theta_t = \lfloor \varepsilon N_t \rfloor$ and preserve GK's rank guarantee. In every round, the circuit tests all $n/2$ pairs and compacts all $n$ rows, including dead rows. The number of fusions and survivors therefore affects the values and flags but not the circuit's structure.

*B.6.2 Correctness of the Update Circuit.* An accepted proof must certify the declared batch rule with the correct threshold and preserve GK's rank guarantee. Assume that $S_{t-1}$ is reachable from $S_0$ under this rule, as certified by the history proof. It then satisfies the invariant, and its live tuples cover exactly the records before the batch. The proof first establishes that the circuit follows the rule for the supplied threshold, then shows that the threshold is correct. This order avoids assuming that the prover's supplied count is valid.

**The union and initialization.** The openings and sorted-copy check determine the old and fresh tuples in $(e, \tau)$ order, with dummies last. Old tuples have positions at most $N_{t-1}$, and fresh tuples have positions $N_{t-1} + 1, \ldots, N_t$. These checked fields distinguish the two groups without assuming that the supplied $N_{t-1}$ is correct. Induction along each directional scan shows that every fresh tuple receives exactly the neighbor bounds of Phase 2. Old tuples retain their fields, and each fresh tuple covers one record. Together, these checks determine $G^{(0)}$.

**The rounds.** Fix the input table of a round. The checked comparisons identify the newer tuple in each pair and determine whether the pair fuses. Only live rows participate in a fusion. If a left tuple $a$ fuses into a right tuple $b$, its coverage is preserved because

$$g_1'(b) + g_2(b) + 1 = \big(g_1(a) + g_2(a) + 1\big) + \big(g_1(b) + g_2(b) + 1\big);$$

the other direction is symmetric. A pair that does not fuse retains both tuples. Since the pairs are disjoint, no tuple transfers its coverage twice in the same round.

The prefix counts assign the $\kappa$ live rows, in order, to slots $1, \ldots, \kappa$ and the dead rows to the remaining slots. Every destination is

unique. Each compaction element carries its round and destination, so the same-rows check moves the complete tuple, including its liveness bit, to the assigned slot. The check rules out moving a tuple to another round, reviving an absorbed tuple, or swapping survivors. Fusion preserves each survivor's $(e, \tau)$, so the next table has a sorted live prefix with unchanged total coverage. By induction, all $L_f$ rounds follow the rule for the supplied threshold. The check that slot $s' + 1$ is dead places every survivor in the first $s'$ slots. Replacing dead rows with dummies therefore loses no coverage, and the output opening binds the result to $S_t$.

**The threshold.** The profile's bounds ensure that every coverage and sum above stays below $|\mathbb{F}|$, without relying on the supplied count. The reachable $S_{t-1}$ covers fewer records than the bound on $N$, which lies far below $|\mathbb{F}|$. The batch adds at most $m$ records, fusion only transfers coverage, and every uncertainty field is either an old field or an old tuple's $g + \Delta$. Together with the range check on $\theta_t$, these bounds make the identities and comparisons valid over the integers. In particular, $\text{mass}(S_t) = \text{mass}(S_{t-1}) + n_t$. Combining this with the checked identity $N_t = N_{t-1} + n_t = \text{mass}(S_t)$ gives $N_{t-1} = \text{mass}(S_{t-1})$, the true number of earlier records. The checked division $N_t = \varepsilon^{-1}\theta_t + r_t$ with $0 \le r_t < \varepsilon^{-1}$ also holds over the integers, and hence $\theta_t = \lfloor \varepsilon N_t \rfloor$.

**The rank guarantee.** With the threshold established, the invariant is preserved. The old tuples' bounds remain valid because $\theta_t$ never decreases. Fresh tuples inherit bounds from their old neighbors. Each allowed fusion checks the changed side against $\theta_t$ while leaving the other side and the survivor's uncertainty unchanged. These are the two-sided insertion and fusion rules of [33], which preserve the rank bounds. For the bracketing tuples of a query (Appendix B.6.3), the rank of $q$ therefore lies between $r_{\min}$ and $r_{\max}$. The bounds differ by at most $2\theta_t$, so the released midpoint has error at most $\theta_t \le \varepsilon N_t$.

**Completeness.** An honest batch admits a witness if its survivors fit in $s'$ slots and the stream stays within the profile's bound. Its sorted union, initialized fields, fusion decisions, and compactions then satisfy every check. The circuit does not require the $L_f$ rounds to reach a fusion fixed point.

*B.6.3 Query Circuit.* A rank query for a value $q$ releases the midpoint of two rank bounds. Let $e_i \le q < e_{i+1}$ be the stored values that bracket $q$, and let $P = \sum_{z \le i} \big(g_1(e_z) + g_2(e_z) + 1\big)$ be the coverage of the tuples up to $e_i$. The rank of $q$ then lies between

$$r_{\min} = P - g_2(e_i) - \Delta_2(e_i), \qquad r_{\max} = P + g_1(e_{i+1}) + \Delta_1(e_{i+1}),$$

with a missing term taken as zero at either end. The circuit rounds their midpoint down, $res = \lfloor (r_{\min} + r_{\max})/2 \rfloor$, as in Section 7.4. The resulting error is at most $\theta_t \le \varepsilon N_t$ (Appendix B.6.2). To certify the answer, the circuit opens $S_t$ and uses a range-checked difference to flag each tuple with $e \le q$. It sums the flagged coverage into $P$ and uses the number of flags to select the bracketing pair. The update circuit already certifies the value order of $S_t$. The query circuit uses $\Theta(s')$ rows.

# C EXPERIMENTAL DETAILS

*Dataset sensitivity.* Proving time varies by at most 1.1% across the four datasets (Table 5), consistent with the fixed circuit sizes discussed in Section 8.4. The spread is 0.5–0.6% for Misra–Gries and Count-Min, 0.8% for Bloom and KMV, and 1.1% for HyperLogLog and GK. GK has the largest absolute difference, 284 s, but this is small relative to its roughly 25 300 s proving time. Uniform keys give the shortest time for every sketch. AOL gives the longest for all but Misra–Gries, where Netflix takes 2 s more.

**Table 5: Proving time of zkSketch with different datasets.**

| | prover time (s) | | | | |
|---|---|---|---|---|---|
| Family | Uniform | Gaussian | Netflix | AOL | spread |
| CM | 10 177 | 10 236 | 10 211 | 10 237 | 0.6% |
| HLL | 10 481 | 10 495 | 10 528 | 10 592 | 1.1% |
| MG | 3 774 | 3 790 | 3 793 | 3 791 | 0.5% |
| GK | 25 268 | 25 362 | 25 326 | 25 552 | 1.1% |
| Bloom | 10 297 | 10 336 | 10 337 | 10 380 | 0.8% |
| KMV | 3 017 | 3 028 | 3 028 | 3 041 | 0.8% |